CHAPTER 4 • REAL-WORLD PATTERNS ~15 Min

Student

Chapter 3 showed me each Snowflake mechanism individually - Snowpipe, Time Travel, cloning, clustering, VARIANT, RBAC. But real production systems combine these into named patterns. What are the standard patterns and how do they fit together?

Claude • AI Assistant

Chapter 4 is the composition chapter. We take the foundational and advanced features and shape them into the **six production patterns** every senior Snowflake engineer builds and maintains: **Streams** for change-data-capture without external tools; **Tasks and DAGs** for scheduling and orchestration inside Snowflake; **UDFs** for reusable logic and in-database ML scoring; **Stored Procedures** for multi-step workflows with control flow; **Search Optimization Service** for point-lookup performance at scale; and **Python Connector plus Snowpark** for programmatic access.

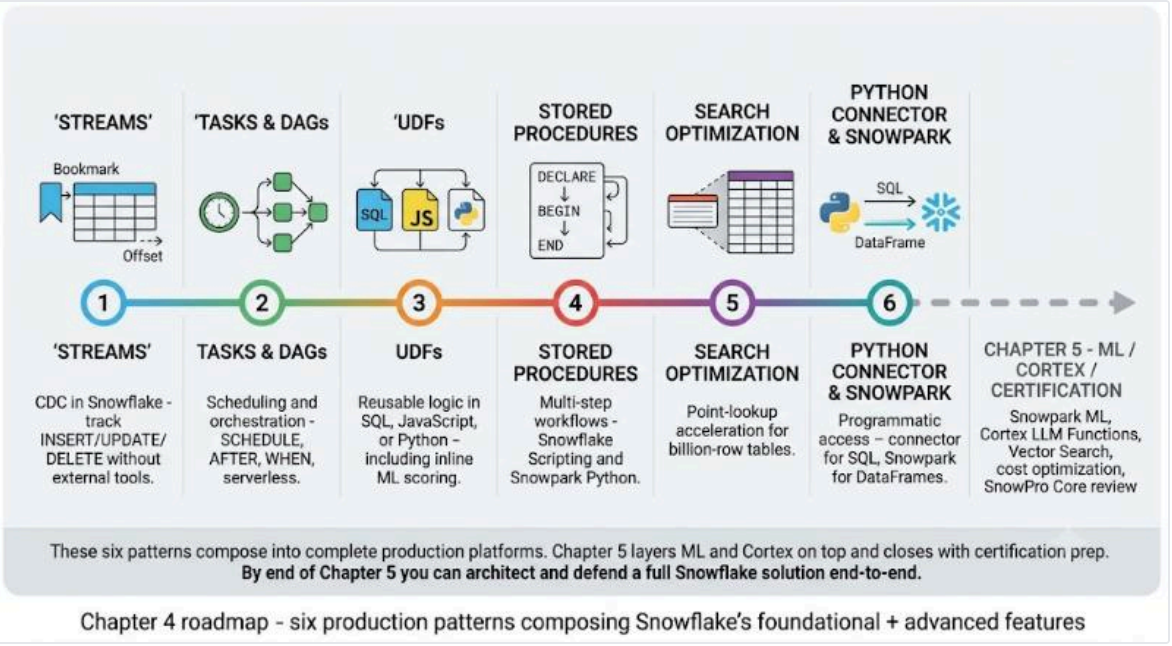
Together these six patterns eliminate most external tooling for Snowflake-centric data platforms - no more Airflow just for warehouse jobs, no more shell scripts, no more separate Python ML serving stack.

The connective tissue with earlier chapters: **Streams depend on Time Travel** (3.2); **Tasks often orchestrate stored procedures that consume Streams from Snowpipe-fed landing tables** (3.1); **Python UDFs use IMPORTS from internal stages** with a familiar `PUT ... @stage` command (Chapter 2); **Search Optimization complements clustering** (3.4) and materialized views (3.4).

Nothing in this chapter is orthogonal to what came before - it's all composition of familiar mechanisms into patterns you can reach for by name.

This chapter is heavy on **runnable, complete examples**. You should finish Chapter 4 able to sketch, on a whiteboard, the end-to-end architecture of a production Snowflake data platform - from event landing to ML scoring to dashboard - with correct feature choices at each step and rough cost estimates for each.

The end-of-chapter capstone asks you to build exactly that platform for Mango Mart. Chapter 5 will then bring ML and Cortex into the same picture.



Chapter 4 roadmap - six production patterns composing Snowflake's foundational + advanced features

Chapter 4 at a glance - what each subsection delivers

Subsection	Title	Core question answered	You can now...
4.1	Streams - CDC in Snowflake	How do I track and process changes without external CDC tools?	Build continuous change-data-capture pipelines and SCD Type 2 dimensions
4.2	Tasks & DAGs	How do I schedule and orchestrate SQL work inside Snowflake?	Replace external orchestrators for Snowflake-internal ETL
4.3	User-Defined Functions	How do I encapsulate reusable logic - including ML scoring?	Write SQL, JavaScript, and Python UDFs and call them from any query
4.4	Stored Procedures	How do I run multi-statement workflows with control flow?	Author Snowflake Scripting and Snowpark Python SPs called by tasks
4.5	Search Optimization Service	How do I accelerate point lookups on huge tables?	Enable SOS on the right columns and reason about its cost/benefit

Subsection	Title	Core question answered	You can now...
4.6	Python Connector & Snowpark	How do I access Snowflake programmatically from Python?	Use the connector for SQL, Snowpark for DataFrame-heavy pipelines
4.7	Chapter Review	How do the six patterns compose into a full production platform?	Build a complete real-time analytics platform end-to-end

sql — Chapter 4 preview - the six patterns in a single condensed script

```sql
-- 4.1 STREAM: track changes to a Snowpipe landing table
CREATE OR REPLACE STREAM raw.strm_new_orders
  ON TABLE raw.raw_orders_stream APPEND_ONLY = TRUE;

-- 4.3 UDF: inline-optimized SQL UDF for reusable business logic
CREATE OR REPLACE FUNCTION utils.gst_amount(net NUMBER)
  RETURNS NUMBER LANGUAGE SQL AS $$ net * 0.18 $$;

-- 4.4 STORED PROCEDURE: multi-statement MERGE with logging
CREATE OR REPLACE PROCEDURE analytics.merge_new_orders()
  RETURNS VARCHAR LANGUAGE SQL AS
$$ BEGIN
     MERGE INTO analytics.fact_orders t USING raw.strm_new_orders s
     ON t.order_id = s.raw_event:order_id::STRING
     WHEN NOT MATCHED THEN INSERT (order_id) VALUES
(s.raw_event:order_id::STRING);
     RETURN 'MERGED ' || SQLROWCOUNT || ' rows';
   END; $$;

-- 4.2 TASK: schedule the SP, skip empty runs via WHEN
CREATE OR REPLACE TASK ops.task_merge_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.strm_new_orders')
AS CALL analytics.merge_new_orders();
ALTER TASK ops.task_merge_orders RESUME;

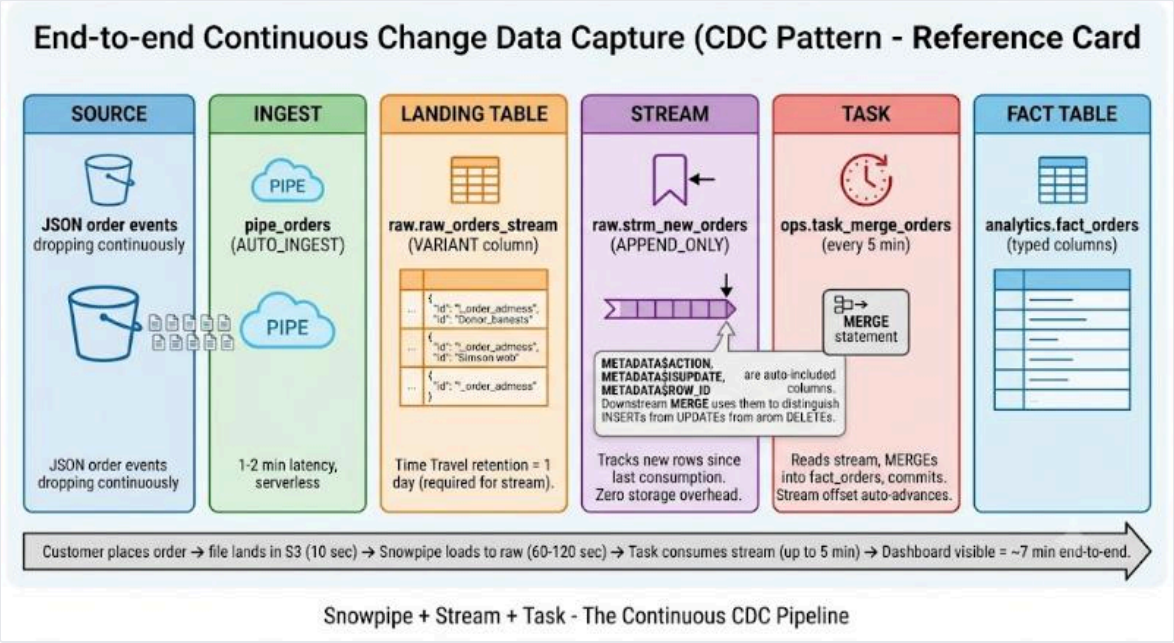
-- 4.5 SEARCH OPTIMIZATION: accelerate point lookups by order_id
ALTER TABLE analytics.fact_sales_transaction
  ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id);

-- 4.6 SNOWPARK (from Python client): compute stays in Snowflake
```

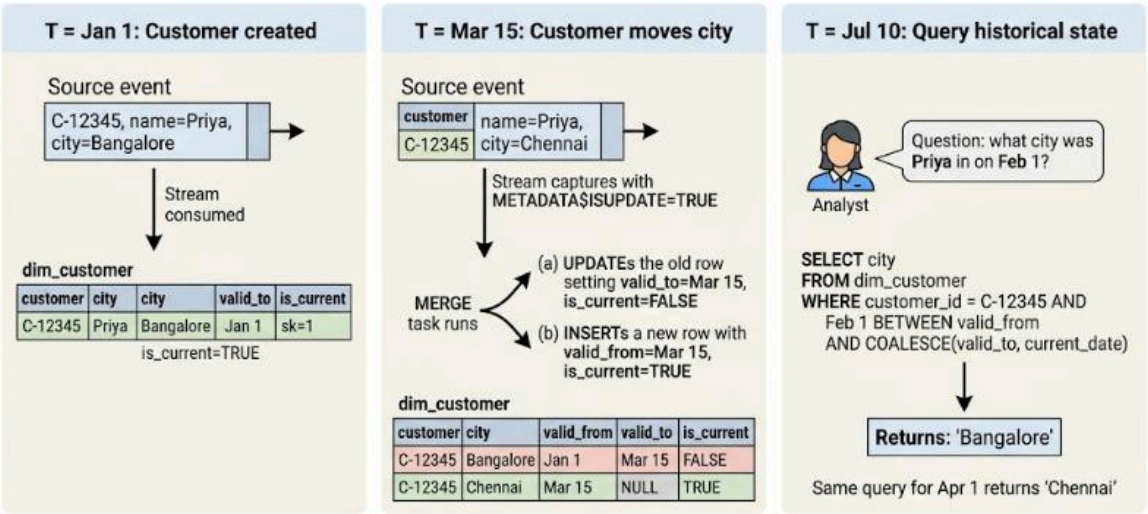
```
-- from snowflake.snowpark import Session
-- session = Session.builder.configs({...}).create()
--
session.table('fact_orders').group_by('customer_id').agg({'net_revenue':'sum'
}).show()
```

STREAMS - CHANGE DATA CAPTURE IN SNOWFLAKE

Claude • AI Assistant



Snowpipe + Stream + Task - The Continuous CDC Pipeline



SCD Type 2 with Streams - How Historical Dimension Rows Are Maintained

sql — Streams end-to-end - create, consume, MERGE, and monitor

```sql
-- 1. Prereq: source table must have Time Travel retention >= 1 day.
-- (Default is 1 day, so usually no action needed.)
SELECT table_name, retention_time
FROM   mango_mart.information_schema.tables
WHERE  table_schema = 'RAW' AND table_name = 'RAW_ORDERS_STREAM';

-- 2. Create an APPEND_ONLY stream on the Snowpipe landing table.
CREATE OR REPLACE STREAM mango_mart.raw.strm_new_orders
  ON TABLE mango_mart.raw.raw_orders_stream
  APPEND_ONLY = TRUE
  COMMENT = 'CDC stream feeding analytics.fact_orders';

-- 3. Preview - see what the stream currently exposes.
-- Read-only; does NOT advance the offset.
SELECT raw_event:order_id::STRING AS order_id,
       raw_event:customer.id::INT AS customer_id,
       METADATA$ACTION,
       METADATA$ISUPDATE
FROM   mango_mart.raw.strm_new_orders
LIMIT  10;

-- 4. Consume the stream inside a MERGE - offset advances on success.
MERGE INTO mango_mart.analytics.fact_orders t
USING (
  SELECT raw_event:order_id::STRING             AS order_id,
         raw_event:customer.id::INT             AS customer_id,
         raw_event:paid_at::TIMESTAMP_TZ        AS paid_at,
```

```
        raw_event:total_amount::NUMBER(12,2)    AS total_amount,
        METADATA$ACTION                          AS action
  FROM   mango_mart.raw.strm_new_orders
  WHERE  METADATA$ACTION = 'INSERT'
) s
ON  t.order_id = s.order_id
WHEN MATCHED     THEN UPDATE SET paid_at = s.paid_at, total_amount =
s.total_amount
WHEN NOT MATCHED THEN INSERT (order_id, customer_id, paid_at, total_amount)
                      VALUES (s.order_id, s.customer_id, s.paid_at,
s.total_amount);

-- 5. Verify the stream is now empty (offset moved forward).
SELECT SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_orders') AS has_data;

-- 6. Monitor - see the current offset timestamp.
SELECT SYSTEM$STREAM_GET_TABLE_TIMESTAMP('mango_mart.raw.strm_new_orders');
```

sql — SCD Type 2 MERGE driven by a Stream - the textbook pattern

```
-- Source: raw.raw_customer_updates (Snowpipe landing).
-- Target: retail.dim_customer with SCD Type 2 columns.

-- Dimension table shape (SCD2 columns highlighted).
CREATE OR REPLACE TABLE mango_mart.retail.dim_customer (
  customer_sk   NUMBER AUTOINCREMENT START 1 INCREMENT 1,
  customer_id   VARCHAR(50)          NOT NULL,
  name          VARCHAR(200),
  email         VARCHAR(200),
  city          VARCHAR(100),
  valid_from    TIMESTAMP_LTZ        NOT NULL,
  valid_to      TIMESTAMP_LTZ,
  is_current    BOOLEAN              DEFAULT TRUE
);

-- Stream on the raw updates (standard - we need UPDATE semantics).
CREATE OR REPLACE STREAM mango_mart.raw.strm_customer_updates
  ON TABLE mango_mart.raw.raw_customer_updates;

-- The SCD Type 2 MERGE - two-step: close old, insert new.
-- Step A: close out old rows for customers whose data changed.
UPDATE mango_mart.retail.dim_customer t
SET    valid_to   = CURRENT_TIMESTAMP,
       is_current = FALSE
WHERE  t.is_current = TRUE
```

```
  AND  t.customer_id IN (
         SELECT DISTINCT raw_event:customer_id::STRING
         FROM   mango_mart.raw.strm_customer_updates
         WHERE  METADATA$ISUPDATE = TRUE OR METADATA$ACTION = 'INSERT'
       );

-- Step B: insert new rows for all changes (new customers AND new versions).
INSERT INTO mango_mart.retail.dim_customer
  (customer_id, name, email, city, valid_from, valid_to, is_current)
SELECT raw_event:customer_id::STRING,
       raw_event:name::STRING,
       raw_event:email::STRING,
       raw_event:city::STRING,
       CURRENT_TIMESTAMP,
       NULL,
       TRUE
FROM   mango_mart.raw.strm_customer_updates
WHERE  METADATA$ACTION = 'INSERT'
  AND  (METADATA$ISUPDATE = TRUE OR METADATA$ISUPDATE = FALSE);

-- After both DMLs commit, the stream offset advances past these changes.

-- Query: current state
SELECT customer_id, name, city FROM mango_mart.retail.dim_customer
WHERE  customer_id = 'C-12345' AND is_current = TRUE;

-- Query: historical state on a specific date (point-in-time)
SELECT customer_id, name, city FROM mango_mart.retail.dim_customer
WHERE  customer_id = 'C-12345'
  AND  '2026-02-01'::DATE BETWEEN valid_from AND COALESCE(valid_to,
CURRENT_TIMESTAMP);
```

Student

We covered Snowpipe for continuous ingest and Time Travel for historical queries. Now I hear about Streams for Change Data Capture. What is a Stream and how does it differ from just re-reading the table?

Claude · AI Assistant

A **Stream** is a Snowflake object that tracks **which rows have changed** in a source table since the last time you consumed the stream. It gives you an ordered, incrementally-

consumed view of INSERTs, UPDATEs, and DELETEs - exactly what you need to build downstream pipelines that process only new/changed data instead of re-scanning full tables.

Streams work by leveraging Snowflake's Time Travel history under the hood: they're essentially a pointer into the change log that automatically advances as you consume it.

WHAT A STREAM ACTUALLY IS

Student

Is it a copy of the changes?

Claude • AI Assistant

CONCEPT CARD

No copies. A **Stream** is a lightweight metadata object that stores an **offset** - a specific point in the source table's Time Travel history.

When you SELECT from the stream, Snowflake computes the difference between the current state and the offset, exposing it as a virtual result set with three extra columns:

- `METADATA$ACTION` - `'INSERT'` or `'DELETE'` (updates appear as a DELETE + INSERT pair).
- `METADATA$ISUPDATE` - `TRUE` if the DELETE/INSERT pair is actually a single logical UPDATE, `FALSE` otherwise.
- `METADATA$ROW_ID` - a unique row identifier used to correlate the two rows of an update pair.

When you consume the stream inside a DML statement (INSERT, MERGE, UPDATE, DELETE) that succeeds and commits, the stream's offset automatically advances to the current point - and the changes you just processed are no longer visible to that stream. This makes streams **exactly-once** by design.

Student

So the source table must have Time Travel enabled?

Claude • AI Assistant

Yes - a source table must have `DATA_RETENTION_TIME_IN_DAYS >= 1` for streams to work. The stream reads Time Travel to compute changes. If your source has retention set to 0,

streams will not work. This is one of the connections back to Chapter 3.

THE STREAM LIFECYCLE

Student

Walk me through one full cycle.

Claude • AI Assistant

CONCEPT CARD

Every stream follows the same rhythm:

1. Create the stream pointing at a source table. The initial offset is set to CURRENT_TIMESTAMP.

2. Data changes in the source - INSERTs, UPDATEs, DELETEs happen from any source (Snowpipe, batch loads, direct DML).

3. SELECT from the stream to preview what changed since the offset. Read-only; offset does NOT advance.

4. Consume the stream inside a successful DML statement (typically a MERGE). Offset advances to the timestamp of that DML.

5. Subsequent SELECTs show only new changes since the DML commit.

The key mental model: the stream doesn't process anything itself. It's a **change ledger with a pointer**; downstream DML consumes and advances it.

THE CANONICAL CDC PATTERN

Student

Show me the whole pattern end to end.

Claude • AI Assistant

EXAMPLE CARD

Mango Mart wants near-real-time propagation of order events from a raw landing table into an analytics fact table.

Setup:

- `raw.raw_orders_stream` - VARIANT landing table filled by Snowpipe (Chapter 3).

- `analytics.fact_orders` - typed fact table for BI.
- `raw.strm_new_orders` - Stream on `raw_orders_stream` capturing new INSERTs.
- `ops.task_merge_orders` - Task that runs every 5 minutes, consumes the stream, MERGEs into `fact_orders` .

Every 5 minutes:

1. Snowpipe has loaded 300 new order events into `raw_orders_stream` .
2. Task fires. Runs a MERGE that USING the stream, INSERTs new rows into `fact_orders` .
3. MERGE commits. Stream offset advances - those 300 rows no longer visible to the stream.
4. If no new events arrive, the next Task fires against an empty stream and does nothing (cheap - just a metadata check).

Zero double-processing, zero missed rows, no need to track state yourself.

SCD TYPE 2 WITH STREAMS

Student

What about slowly-changing dimensions?

Claude • AI Assistant

CONCEPT CARD

Streams enable a clean **Slowly Changing Dimension Type 2** implementation. Recall from your Kimball training (RC-753): SCD Type 2 keeps history by inserting a new row each time an attribute changes, with `valid_from` / `valid_to` timestamps and a `is_current` flag.

Pattern:

- Source table `raw.raw_customer_updates` receives new customer snapshots continuously.
- Stream `strm_customer_updates` on it captures changes.
- Task runs a MERGE:
- For rows with `METADATA$ISUPDATE = TRUE` : UPDATE existing `dim_customer` row's `valid_to = current_timestamp` and `is_current = FALSE` ; then INSERT a new row with new attributes, `valid_from = current_timestamp` , `is_current = TRUE` .
- For rows with `METADATA$ACTION = 'INSERT'` (brand new customer): straight INSERT.

This is the textbook SCD Type 2, driven by a stream. No batch job comparing full snapshots - only the actual changes are processed.

APPEND_ONLY STREAMS FOR HIGH-VOLUME INSERTS

Student

What if I only care about inserts?

Claude • AI Assistant

CONCEPT CARD

For pure event-append tables (logs, telemetry, orders that never get updated) use `APPEND_ONLY` streams:

sql — Code Example

```
CREATE OR REPLACE STREAM raw.strm_events_append
  ON TABLE raw.event_log
  APPEND_ONLY = TRUE;
```

APPEND*ONLY streams skip processing update and delete history, making them significantly faster and cheaper to consume on large tables. Since most streaming-ingest tables are insert-only (they hold raw events that are never modified in place), APPEND*ONLY is the right default for the Snowpipe -> Stream pattern.

MULTIPLE CONSUMERS, ONE SOURCE

Student

Can I have multiple streams on the same source?

Claude • AI Assistant

Yes - and this is one of the key advantages over external CDC. Multiple streams on the same source each maintain their own independent offset. This lets several downstream consumers each track changes at their own pace without interfering:

- `strm_orders_to_fact` consumed by the analytics MERGE task every 5 min.
- `strm_orders_to_ops_alert` consumed by an alerting task every 30 sec.
- `strm_orders_to_ml_features` consumed by a nightly ML feature pipeline.

Each stream advances independently. No coordination needed.

STREAM STALENESS AND RETENTION

Student

What happens if I never consume a stream?

Claude • AI Assistant

⚠ **WARNING**

Streams can become **stale**. A stream's offset points to a specific past time; if that time falls outside the source table's `DATA_RETENTION_TIME_IN_DAYS` window (extended by any downstream stream dependency), the stream cannot reconstruct history and becomes stale.

Stale streams return empty results and must be recreated.

Snowflake **automatically extends** the source's Time Travel retention to at least the age of the oldest unconsumed stream (up to a maximum of 14 days by default). This gives you a safety window - but a stream left unconsumed for weeks WILL go stale.

Monitor with `SYSTEM$STREAM_HAS_DATA` and `SYSTEM$STREAM_GET_TABLE_TIMESTAMP` .

STREAMS ON VIEWS

Student

Can streams work on views too?

Claude • AI Assistant

CONCEPT CARD

Yes - a stream can be created on a **view** that references one or more base tables (with the CHANGE_TRACKING property enabled on those bases). This lets you capture changes to a joined/filtered projection of source tables.

Useful when the downstream consumer wants only a subset or a specific join shape without repeating the JOIN logic in every consumer.

The catch: streams on views require the base tables to have change tracking explicitly enabled (`ALTER TABLE ... SET CHANGE_TRACKING = TRUE`) which uses extra storage for the change metadata. Enable only where you need it.

ANALOGY

A Stream is a **bookmark in a book that automatically moves as you read**. The book (source table) keeps getting new pages (changes).

Your bookmark (stream offset) shows you which pages you haven't read yet. When you 'read' (consume the stream in a DML), the bookmark moves forward.

Multiple readers can each keep their own bookmark in the same book. The publisher (Time Travel) keeps enough old pages around that your bookmark never points into a hole.

REAL WORLD APPLICATION

A logistics client had a legacy CDC tool (based on triggers and staging tables) taking about 4 hours per night to propagate 50 million daily events from raw to analytics. Replaced with a Snowpipe + APPEND_ONLY Stream + 5-minute Task.

New end-to-end latency: about 6 minutes from event arrival to analytics-ready. Nightly batch window freed up entirely.

Operational complexity dropped drastically - no more trigger maintenance, no more custom offset tables. Streams are the piece that closes the loop between Snowpipe's continuous ingest and structured analytics.

WARNING

Consuming a stream OUTSIDE a DML - such as SELECTing it into a session variable or exporting it via a Python job - does NOT advance the offset. Only DML statements that consume the stream and commit successfully advance it.

To force-advance without doing real work: `CREATE OR REPLACE TEMPORARY TABLE tmp AS SELECT * FROM my_stream;` counts as a DML consumption.

COMMON PITFALL / MISTAKE

Creating a stream on a source table with `DATA_RETENTION_TIME_IN_DAYS = 0`. The stream is created but immediately stale - it has no Time Travel history to read from. Always ensure sources have at least 1 day of retention (default) before creating streams.

EXAM TIP CARD

SnowPro Core will test: (1) streams track INSERT/UPDATE/DELETE via METADATA$ACTION and METADATA$ISUPDATE. (2) Offset advances only on successful DML consumption.

(3) APPEND_ONLY streams are faster and cheaper for insert-only tables. (4) Multiple streams on the same source each maintain independent offsets.

(5) Streams depend on source Time Travel and can go stale.

PRACTICE QUIZ

You create a stream on `raw.event_log` (a Snowpipe target). Between two consumptions, 1,000 rows are inserted and 200 of those are later updated. What does the standard stream show at consumption time?

A) 1,000 inserted rows only

B) 200 update pairs (DELETE + INSERT) and 800 pure INSERTs

C) 800 INSERT rows (net) - the 200 updated rows collapse into their final state

D) 1,200 rows total (200 pairs + 800 inserts + 200 originals)

▼ Reveal Correct Answer

*Answer: C. A standard stream shows the NET change per row.

If a row was inserted and then updated within the same stream window, the net result is one INSERT with the final value. B would be correct if we asked about intermediate history; for exactly-once downstream MERGE, C is what you get.*

EXERCISE

For Mango Mart, design a stream-based SCD Type 2 pipeline for `dim_customer`. Source is `raw.raw_customer_updates` (fed by Snowpipe).

Include: (a) stream definition, (b) task/MERGE that closes old rows and inserts new versions with valid*from/valid*to, (c) query to see current customer state, (d) query to see historical customer state at a specific date.

AI DEEP DIVE PROMPT

Ask an AI: '*Explain how a Snowflake Stream works internally in relation to Time Travel, and show a complete CDC pattern from Snowpipe landing table through Stream to a downstream MERGE task, including SCD Type 2 handling and multiple concurrent consumers.*'

SUMMARY CARD

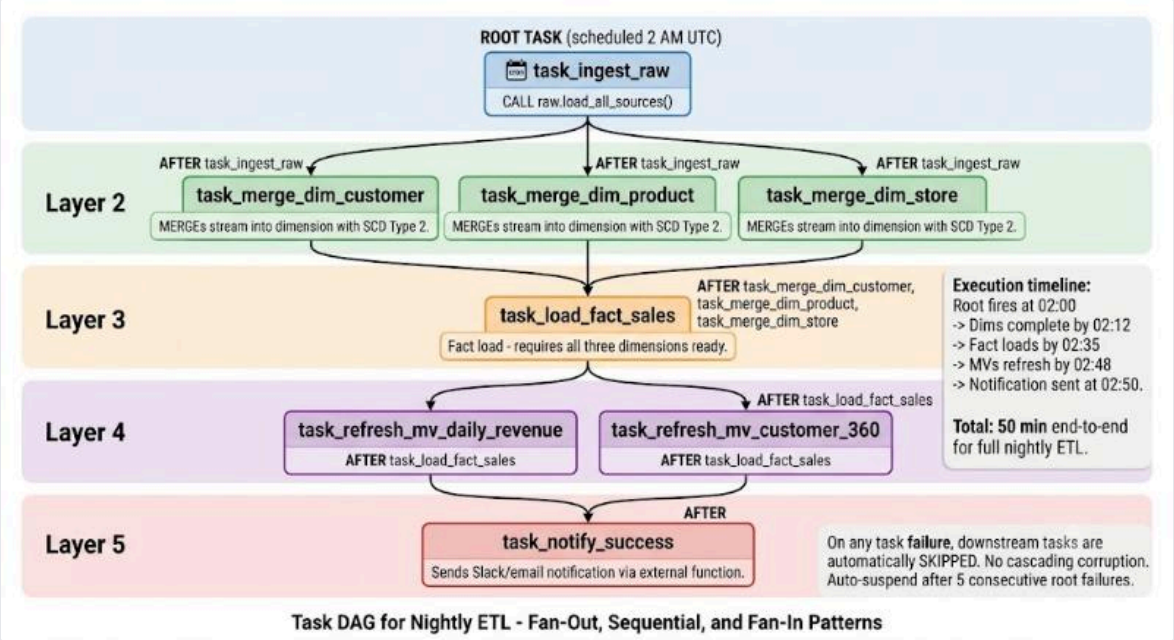
Streams are lightweight offset-tracking objects that expose the incremental changes to a source table since last consumption. Three types: STANDARD (all changes), APPEND*ONLY (inserts only, faster for logs), INSERT*ONLY (for external/directory tables).

Consumed inside DML statements which advance the offset automatically. Depend on source Time Travel; auto-extend retention up to 14 days but can go stale.

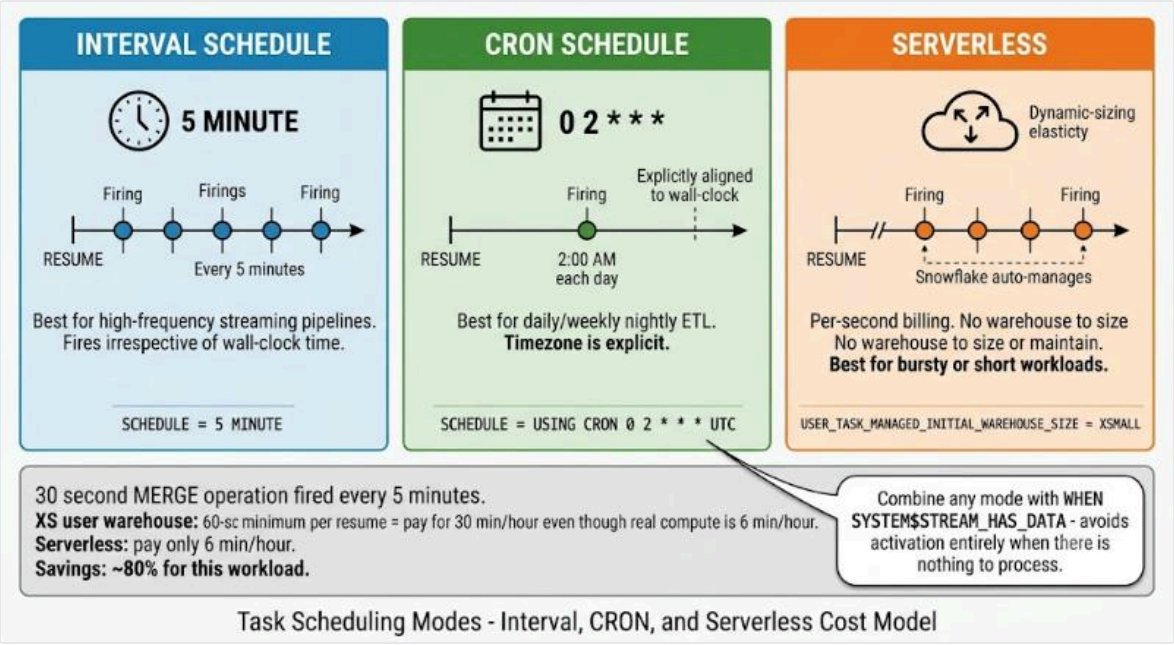
Enable Snowpipe -> Stream -> Task pipelines and clean SCD Type 2 implementations without external CDC tools.

TASKS AND DAGS - SCHEDULING AND ORCHESTRATION

Claude • AI Assistant



Task DAG for Nightly ETL - Fan-Out, Sequential, and Fan-In Patterns



Task Scheduling Modes - Interval, CRON, and Serverless Cost Model

sql — Tasks + Streams - the standard continuous CDC combo

```sql
-- Stream that captures new orders (from 4.1).
CREATE OR REPLACE STREAM mango_mart.raw.strm_new_orders
  ON TABLE mango_mart.raw.raw_orders_stream
  APPEND_ONLY = TRUE;
```

```
-- Task that consumes it every 5 minutes ONLY if there is data.
-- WHEN clause is evaluated without activating the warehouse.
CREATE OR REPLACE TASK mango_mart.ops.task_merge_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_orders')
AS
  MERGE INTO mango_mart.analytics.fact_orders t
  USING (
    SELECT raw_event:order_id::STRING              AS order_id,
           raw_event:customer.id::INT              AS customer_id,
           raw_event:paid_at::TIMESTAMP_TZ         AS paid_at,
           raw_event:total_amount::NUMBER(12,2)    AS total_amount
    FROM mango_mart.raw.strm_new_orders
    WHERE METADATA$ACTION = 'INSERT'
  ) s
  ON  t.order_id = s.order_id
  WHEN NOT MATCHED THEN
    INSERT (order_id, customer_id, paid_at, total_amount)
    VALUES (s.order_id, s.customer_id, s.paid_at, s.total_amount);

-- Activate the schedule.
ALTER TASK mango_mart.ops.task_merge_orders RESUME;

-- Manually fire once for testing (bypasses schedule).
EXECUTE TASK mango_mart.ops.task_merge_orders;

-- Monitor: recent task executions with state and duration.
SELECT name, state, scheduled_time, completed_time,
       DATEDIFF('SECOND', query_start_time, completed_time) AS runtime_sec,
       error_message
FROM   TABLE(INFORMATION_SCHEMA.TASK_HISTORY(
         SCHEDULED_TIME_RANGE_START => DATEADD('HOUR', -6,
CURRENT_TIMESTAMP)))
WHERE  name = 'TASK_MERGE_ORDERS'
ORDER  BY scheduled_time DESC;
```

sql — Task DAG - a nightly ETL pipeline with fan-out and fan-in

```
-- Root task - scheduled at 2 AM UTC daily.
CREATE OR REPLACE TASK mango_mart.ops.task_ingest_raw
  WAREHOUSE = etl_wh
  SCHEDULE = 'USING CRON 0 2 * * * UTC'
AS
```

```
  CALL mango_mart.raw.load_all_sources();

-- Fan-out: three dimension merges run in parallel after ingest.
CREATE OR REPLACE TASK mango_mart.ops.task_merge_dim_customer
  WAREHOUSE = etl_wh AFTER mango_mart.ops.task_ingest_raw AS
  CALL mango_mart.retail.merge_dim_customer();

CREATE OR REPLACE TASK mango_mart.ops.task_merge_dim_product
  WAREHOUSE = etl_wh AFTER mango_mart.ops.task_ingest_raw AS
  CALL mango_mart.retail.merge_dim_product();

CREATE OR REPLACE TASK mango_mart.ops.task_merge_dim_store
  WAREHOUSE = etl_wh AFTER mango_mart.ops.task_ingest_raw AS
  CALL mango_mart.retail.merge_dim_store();

-- Fan-in: fact load waits for ALL three dimensions.
CREATE OR REPLACE TASK mango_mart.ops.task_load_fact_sales
  WAREHOUSE = etl_wh
  AFTER mango_mart.ops.task_merge_dim_customer,
        mango_mart.ops.task_merge_dim_product,
        mango_mart.ops.task_merge_dim_store
AS
  CALL mango_mart.analytics.load_fact_sales();

-- Downstream: refresh materialized views
CREATE OR REPLACE TASK mango_mart.ops.task_refresh_mv
  WAREHOUSE = etl_wh AFTER mango_mart.ops.task_load_fact_sales AS
  CALL mango_mart.analytics.refresh_mvs();

-- Resume in reverse dependency order (leaves first).
ALTER TASK mango_mart.ops.task_refresh_mv          RESUME;
ALTER TASK mango_mart.ops.task_load_fact_sales     RESUME;
ALTER TASK mango_mart.ops.task_merge_dim_store     RESUME;
ALTER TASK mango_mart.ops.task_merge_dim_product   RESUME;
ALTER TASK mango_mart.ops.task_merge_dim_customer  RESUME;
ALTER TASK mango_mart.ops.task_ingest_raw          RESUME;

-- Alternative: resume the whole DAG at once (rooted at ingest_raw).
SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('mango_mart.ops.task_ingest_raw');

-- Monitor: last N DAG runs with total wall time.
SELECT root_task_name, state, run_id,
       DATEDIFF('SECOND', scheduled_time, completed_time) AS total_sec,
       error_message
FROM   TABLE(INFORMATION_SCHEMA.COMPLETE_TASK_GRAPHS(
         ROOT_TASK_NAME => 'TASK_INGEST_RAW',
```

```
        RESULT_LIMIT   => 20))
ORDER  BY scheduled_time DESC;
```

Student

We have Streams for CDC and Snowpipe for ingest. But something has to schedule and orchestrate the MERGEs and any multi-step pipelines. What's Snowflake's answer?

Claude • AI Assistant

Snowflake's answer is **Tasks**. A Task is a scheduled unit of SQL work - a single statement, a stored procedure call, or a chain of them via **DAGs** (Directed Acyclic Graphs of dependent tasks).

Tasks handle the 'when' and 'in what order' of your data pipeline entirely inside Snowflake. No external Airflow, no cron jobs, no Lambda functions needed for scheduled work that lives inside your warehouse.

THE SINGLE-TASK PATTERN

Student

Show me the simplest task.

Claude • AI Assistant

CONCEPT CARD

A **Task** is defined by:

- **A schedule** - either an interval (`SCHEDULE = '5 MINUTE'`) or a CRON expression (`SCHEDULE = 'USING CRON 0 2 * * * UTC'` for 2 AM daily).
- **A warehouse** - either a named user warehouse or serverless (Snowflake-managed).
- **A body** - a single SQL statement or `CALL stored_procedure()` .

Tasks are created in a **SUSPENDED** state by default; you must explicitly `RESUME` them to activate the schedule. This is a safety feature - you can create and test the task's SQL without immediately committing to running it every 5 minutes.

sql — Code Example

```
CREATE OR REPLACE TASK ops.task_merge_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
AS
  MERGE INTO analytics.fact_orders t
  USING raw.strm_new_orders s
  ON t.order_id = s.raw_event:order_id::STRING
  WHEN NOT MATCHED THEN INSERT (...) VALUES (...);

ALTER TASK ops.task_merge_orders RESUME;
```

INTERVAL VS CRON SCHEDULING

Student

When do I use each?

Claude • AI Assistant

CONCEPT CARD

Interval scheduling (`SCHEDULE = '5 MINUTE'`): fires every N minutes starting from RESUME time. Perfect for high-frequency streaming pipelines where the exact wall-clock time doesn't matter - only the recurring interval.

CRON scheduling (`SCHEDULE = 'USING CRON 0 2 * * * UTC'`): fires at wall-clock times matching the CRON expression. Use for daily/weekly/monthly jobs where the exact hour matters (nightly ETL at 2 AM, weekly reports Monday morning).

The **timezone** is part of the expression - always specify it explicitly to avoid daylight-saving-time surprises.

Example CRON expressions Mango Mart might use:

- `'USING CRON 0 2 * * * UTC'` - every day at 2 AM UTC (7:30 AM IST).
- `'USING CRON 0 8 * * MON UTC'` - every Monday 8 AM UTC (weekly rollup).
- `'USING CRON */15 * * * * UTC'` - every 15 minutes (dashboard refresh).

THE TASK DAG - DEPENDENT TASKS

Student

How do I chain tasks?

Claude • AI Assistant

CONCEPT CARD

Snowflake **Task DAGs** let you build a directed acyclic graph of tasks. Instead of a schedule, child tasks specify `AFTER parent_task_name` and fire only when the parent (or all parents) completes successfully.

sql — Code Example

```sql
-- Root task with a schedule
CREATE OR REPLACE TASK ops.task_load_raw
  WAREHOUSE = etl_wh
  SCHEDULE = 'USING CRON 0 2 * * * UTC'
AS CALL raw.load_raw_customer_updates();

-- Child - runs after load_raw completes
CREATE OR REPLACE TASK ops.task_merge_dim
  WAREHOUSE = etl_wh
  AFTER ops.task_load_raw
AS CALL retail.merge_dim_customer();

-- Grandchild - runs after merge_dim
CREATE OR REPLACE TASK ops.task_refresh_mv
  WAREHOUSE = etl_wh
  AFTER ops.task_merge_dim
AS CALL analytics.refresh_customer_mv();

-- Resume all in reverse dependency order (leaves first)
ALTER TASK ops.task_refresh_mv RESUME;
ALTER TASK ops.task_merge_dim RESUME;
ALTER TASK ops.task_load_raw RESUME;
```

Now `task_load_raw` fires every 2 AM. On success, `task_merge_dim` fires. On its success, `task_refresh_mv` fires. On any failure, downstream tasks are skipped. This is a full nightly ETL DAG in ~15 lines of SQL, no Airflow required.

Student

Can a task have multiple parents?

Claude • AI Assistant

Yes - `AFTER parent_a, parent_b, parent_c` waits for all three to succeed. This is how you build fan-in patterns where a downstream task depends on multiple parallel upstream tasks.

Combined with multiple children of a single parent (fan-out), you can build arbitrarily complex DAGs.

SERVERLESS TASKS

Student

What is a serverless task?

Claude • AI Assistant

CONCEPT CARD

Instead of specifying a `WAREHOUSE`, you specify `USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE = 'XSMALL'` (or LARGE, etc.) on the task. Snowflake then manages the compute for you:

- **No warehouse to size or maintain.**
- **Billed per-second** for actual runtime, not warehouse-hours.
- **Dynamic sizing** - Snowflake can adjust warehouse size between runs based on observed load.
- **Best for bursty or variable workloads** where a dedicated warehouse would be idle much of the time.

For a 30-second MERGE that fires every 5 minutes: serverless is much cheaper than an XS warehouse that must resume for each run (60-second minimum billing per resume adds up).

CONDITIONAL EXECUTION - THE WHEN CLAUSE

Student

Can tasks skip themselves when there's nothing to do?

Claude · AI Assistant

CONCEPT CARD

Yes - `WHEN <boolean_expression>` guards a task. If the expression is FALSE at task fire time, the task skips (doesn't consume compute):

sql — Code Example

```sql
CREATE OR REPLACE TASK ops.task_merge_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('raw.strm_new_orders')
AS
  MERGE INTO analytics.fact_orders ...;
```

The `WHEN` clause uses `SYSTEM$STREAM_HAS_DATA` to check the stream. If empty, the task skips - no warehouse activation, no billing. This is the standard pattern for stream-driven tasks: fire every 5 minutes, but only actually do work if the stream has data.

MONITORING TASKS

Student

How do I see what tasks are running?

Claude · AI Assistant

CONCEPT CARD

Two essential views:

- **`TASK_HISTORY`** - every task execution with state (SUCCEEDED, FAILED, SKIPPED), duration, error message, warehouse used. Available for 365 days in ACCOUNT_USAGE.
- **`COMPLETE_TASK_GRAPHS`** - one row per DAG run showing the root task, run start/end, overall state. Great for auditing multi-task pipelines.

For interactive debugging: `SHOW TASKS` lists all tasks with current state; `DESCRIBE TASK` shows definition; `EXECUTE TASK task_name` manually fires a task once regardless of schedule.

FAILURE HANDLING

Student

What happens on failures?

Claude • AI Assistant

⚠ WARNING

Snowflake automatically **suspends a task after 5 consecutive failures**. This is a safety feature to prevent a broken task from firing every 5 minutes indefinitely, generating error alerts and consuming credits.

Once suspended, you must fix the underlying issue and `ALTER TASK ... RESUME` manually.

Change the threshold with `SUSPEND_TASK_AFTER_NUM_FAILURES` at the account or task level. Set to 0 to disable auto-suspend (not recommended for production).

For DAGs: when a task fails, downstream (dependent) tasks are automatically skipped for that run - they wait until the next scheduled root-task fire. This prevents cascading corrupted data through the pipeline.

ANALOGY

Tasks are **cron jobs inside Snowflake, with cron's older ancestor's power**. A single task is a scheduled shell script; a Task DAG is a Makefile - each target depends on previous targets, and the whole thing rebuilds in order when the root fires.

The advantage over external cron/Airflow: the schedule, the SQL, and the audit history all live in Snowflake with no additional infrastructure to run, monitor, or authenticate.

REAL WORLD APPLICATION

A B2B SaaS analytics team had 40+ Airflow DAGs handling nightly loads into Snowflake. When they moved orchestration into Snowflake Tasks (chained by AFTER), they retired the Airflow cluster entirely - saved about $2,400/month in EC2 and eliminated a whole class of failures (Airflow-Snowflake connection timeouts, missing IAM permissions, etc.).

Task DAGs won on TCO. Airflow still made sense for cross-system orchestration (Snowflake + Salesforce + S3 + custom API), but pure Snowflake-internal workflows moved fully in-warehouse.

⚠ WARNING

`ALTER TASK ... RESUME` on a task with `AFTER` parents doesn't do anything if the parent is still SUSPENDED - the chain won't fire.

Always resume tasks in reverse dependency order (leaves first, root last). Or use `SELECT SYSTEM$TASK_DEPENDENTS_ENABLE('root_task_name');` to resume the whole DAG at once.

COMMON PITFALL / MISTAKE

Setting `SCHEDULE` and `AFTER` on the same task. This is a syntax error - a task is either scheduled (root) OR triggered by parent completion (child), never both. If you need both time-based AND dependency-based triggering, split into two tasks: one scheduled, one dependent.

EXAM TIP CARD

SnowPro Core will test: (1) tasks created in SUSPENDED state - must RESUME. (2) SCHEDULE = interval or CRON with timezone.

(3) AFTER for DAG dependencies. (4) WHEN clause with SYSTEM$STREAM*HAS*DATA for stream-driven tasks.

(5) Serverless tasks via USER*TASK*MANAGED*INITIAL*WAREHOUSE_SIZE. (6) Auto-suspend after 5 consecutive failures.

PRACTICE QUIZ

You have a task that MERGEs a stream into a fact table every 5 minutes. Most of the time the stream is empty. You want to avoid warehouse activation when there's nothing to do. Best pattern?

A) Reduce the schedule to every 30 minutes

B) Add a WHEN SYSTEM$STREAM*HAS*DATA('stream_name') clause

C) Change to a serverless task

D) Add an IF check inside the MERGE

▼ Reveal Correct Answer

Answer: B. WHEN clause is evaluated at task fire time WITHOUT activating the warehouse. If FALSE, the task skips and consumes no credits. C helps with per-run cost but doesn't avoid the run itself. D consumes credits to run the IF check.

EXERCISE

For Mango Mart, design a nightly ETL DAG with these steps: (1) load raw customer updates from S3, (2) MERGE into dim_customer with SCD Type 2, (3) recompute customer segment tags, (4) refresh two materialized views, (5) send success notification. Write the task definitions with correct AFTER dependencies.

AI DEEP DIVE PROMPT

> Ask an AI: '*Compare Snowflake Tasks + DAGs against Apache Airflow for orchestrating a data warehouse ETL pipeline. Cover operational complexity, cost, monitoring, and integration with non-Snowflake systems. Recommend when to use each.*'

SUMMARY CARD

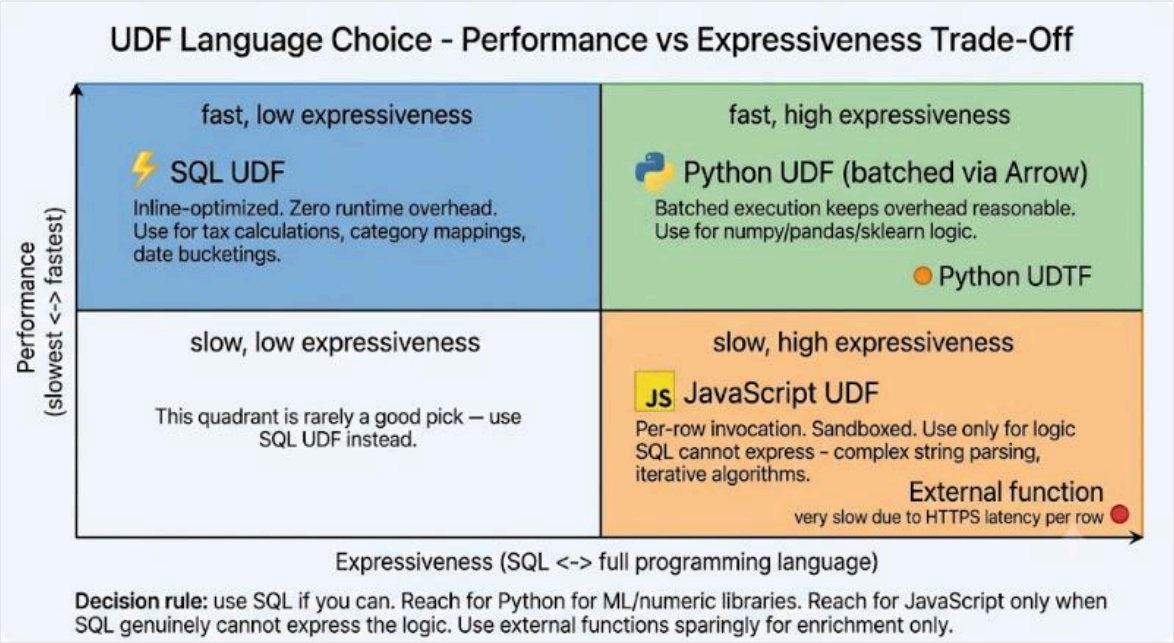
Tasks are Snowflake-native scheduled SQL units - either standalone (SCHEDULE) or chained into **DAGs** (AFTER parent). Created SUSPENDED; must RESUME to activate.

Use a user warehouse or go serverless with USER*TASK*MANAGED*INITIAL*WAREHOUSE*SIZE for bursty workloads. Combine WHEN SYSTEM$STREAM*HAS_DATA with streams to skip empty fires.

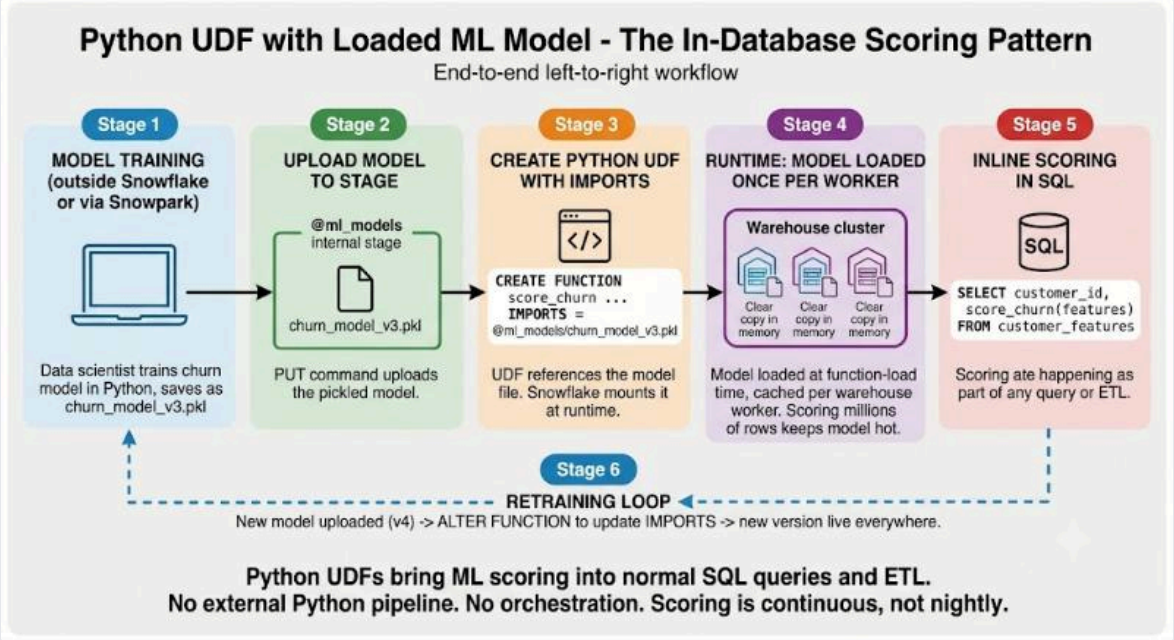
Auto-suspend after 5 consecutive failures. Task DAGs eliminate the need for external orchestrators for Snowflake-internal pipelines.

USER-DEFINED FUNCTIONS - SQL, JAVASCRIPT, AND PYTHON UDFS

Claude · AI Assistant



UDF Language Choice - Performance vs Expressiveness Trade-Off



Python UDF with Loaded ML Model - The In-Database Scoring Pattern

sql — SQL UDF - the fast default; simple and inline-optimized

```sql
-- SQL scalar UDF - a tax calculator used throughout the warehouse.
CREATE OR REPLACE FUNCTION mango_mart.utils.gst_amount(net NUMBER)
  RETURNS NUMBER
  LANGUAGE SQL
AS $$ net * 0.18 $$;

SELECT order_id, net_amount,
       mango_mart.utils.gst_amount(net_amount) AS gst,
       net_amount + mango_mart.utils.gst_amount(net_amount) AS total
FROM   mango_mart.analytics.fact_orders
LIMIT  10;

-- SQL scalar UDF returning a tier label from lifetime spend.
CREATE OR REPLACE FUNCTION mango_mart.utils.customer_tier(ltv NUMBER)
  RETURNS VARCHAR
  LANGUAGE SQL
AS $$ CASE
         WHEN ltv >= 100000 THEN 'GOLD'
         WHEN ltv >=  25000 THEN 'SILVER'
         WHEN ltv >   0     THEN 'BRONZE'
         ELSE 'UNRATED'
      END $$;

-- SQL table function - top N customers per region.
CREATE OR REPLACE FUNCTION mango_mart.analytics.top_customers_by_region(
  region STRING, n INT
)
```

```
  RETURNS TABLE (customer_id STRING, total_spent NUMBER)
  LANGUAGE SQL
AS $$
  SELECT customer_id, SUM(net_revenue) AS total_spent
  FROM   mango_mart.analytics.fact_orders
  WHERE  region_code = region
  GROUP  BY customer_id
  ORDER  BY total_spent DESC
  LIMIT  n
$$;

SELECT * FROM TABLE(mango_mart.analytics.top_customers_by_region('SOUTH',
20));
```

sql — Python UDF with loaded ML model - the in-database scoring pattern

```
-- 1. Upload the model to an internal stage (from SnowSQL locally).
-- PUT file:///local/path/churn_model_v3.pkl @mango_mart.ml.ml_models
--     AUTO_COMPRESS = FALSE OVERWRITE = TRUE;

-- 2. Verify the file landed.
LIST @mango_mart.ml.ml_models PATTERN='.*churn.*';

-- 3. Create the Python UDF referencing the model.
CREATE OR REPLACE FUNCTION mango_mart.ml.score_churn(
    tenure_days INT,
    avg_order_value NUMBER,
    order_count INT,
    days_since_last_order INT
)
  RETURNS NUMBER
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('scikit-learn', 'joblib', 'numpy')
  IMPORTS = ('@mango_mart.ml.ml_models/churn_model_v3.pkl')
  HANDLER = 'score'
AS
$$
import sys, os, joblib
import numpy as np

# Load the model once per Python worker - stays hot for all subsequent rows.
IMPORT_DIR = sys._xoptions.get('snowflake_import_directory')
MODEL = joblib.load(os.path.join(IMPORT_DIR, 'churn_model_v3.pkl'))
```

```
def score(tenure_days, avg_order_value, order_count, days_since_last_order):
    if any(v is None for v in [tenure_days, avg_order_value,
                               order_count, days_since_last_order]):
        return None
    features = np.array([[tenure_days, avg_order_value,
                          order_count, days_since_last_order]])
    proba = MODEL.predict_proba(features)[0][1]
    return float(proba)
$$;

-- 4. Use the UDF inline in any query or ETL.
SELECT customer_id,
       mango_mart.ml.score_churn(tenure_days, avg_order_value,
                                 order_count, days_since_last_order)
         AS churn_probability
FROM   mango_mart.analytics.customer_features_current
ORDER  BY churn_probability DESC
LIMIT  100;
```

Student

Sometimes I need logic that doesn't fit neatly into SQL - date munging, custom string parsing, ML scoring. What are UDFs and when do I use each language?

Claude • AI Assistant

User-Defined Functions (UDFs) let you package logic into a named, reusable function callable from any SQL statement. Snowflake supports UDFs in **SQL**, **JavaScript**, **Python** (via Snowpark), and **Java** (via Snowpark).

Each has its place; choosing wisely balances performance, expressiveness, and maintenance cost. As a rule: **use SQL if you can, JavaScript for row-level string work SQL cannot express, Python when you need libraries or ML.**

SQL UDFS - THE DEFAULT CHOICE

Student

Show me the simplest UDF.

Claude • AI Assistant

CONCEPT CARD

A **SQL UDF** wraps a single SQL expression:

sql — Code Example

```sql
CREATE OR REPLACE FUNCTION utils.gst_amount(net NUMBER)
  RETURNS NUMBER
  LANGUAGE SQL
AS
$$
  net * 0.18
$$;

-- Usage in any SELECT
SELECT order_id, net_amount, utils.gst_amount(net_amount) AS gst
FROM   fact_orders;
```

SQL UDFs are **inline-optimized** - Snowflake's query planner substitutes the function body into the calling query before execution. This means SQL UDFs have essentially **zero runtime overhead**; they're as fast as writing the expression inline everywhere.

For repeated business logic (tax calculations, category mappings, date bucketings), SQL UDFs are the right first choice.

Student

Can SQL UDFs be table-valued?

Claude • AI Assistant

Yes - a SQL UDF can return a table. These are called **SQL table functions** and are used like a table in a FROM clause:

sql — Code Example

```sql
CREATE OR REPLACE FUNCTION analytics.top_customers_by_region(region STRING, n
INT)
  RETURNS TABLE (customer_id STRING, total_spent NUMBER)
  LANGUAGE SQL
AS
$$
  SELECT customer_id, SUM(net_revenue) AS total_spent
```

```
  FROM fact_orders WHERE region_code = region
  GROUP BY customer_id
  ORDER BY total_spent DESC
  LIMIT n
$$;

-- Usage
SELECT * FROM TABLE(analytics.top_customers_by_region('SOUTH', 10));
```

JAVASCRIPT UDFS - FOR WHAT SQL CANNOT EXPRESS

Student

When do I reach for JavaScript?

Claude • AI Assistant

CONCEPT CARD

JavaScript UDFs are the right tool when your logic is **procedural per-row work** that SQL cannot cleanly express: complex string parsing, custom regex logic, iterative algorithms, conditional branching that would be a mess in CASE statements.

sql — Code Example

```
CREATE OR REPLACE FUNCTION utils.parse_customer_ref(ref STRING)
  RETURNS OBJECT
  LANGUAGE JAVASCRIPT
AS
$$
  // Reference format: 'MM-<region>-<yyyy>-<customer_num>'
  // Return an object with the parsed components.
  if (!REF) return { valid: false };
  const parts = REF.split('-');
  if (parts.length !== 4) return { valid: false, raw: REF };
  return {
    valid: true,
    region: parts[1],
    year: parseInt(parts[2], 10),
    customer_num: parseInt(parts[3], 10)
  };
$$;

-- Usage
```

```
SELECT ref, utils.parse_customer_ref(ref):region::STRING AS region
FROM   staging_refs;
```

Important quirks: JavaScript UDF **parameter names are UPPERCASED** inside the function body (Snowflake convention - the parameter `ref` becomes `REF`). The return type can be a scalar or an OBJECT/ARRAY (VARIANT).

Student

What are the limits?

Claude • AI Assistant

⚠ **WARNING**

JavaScript UDFs run in a **sandboxed environment**:

- No network I/O (no HTTP calls, no external service access).
- No file system access.
- Limited memory per invocation.
- ES5+ JavaScript syntax; no npm packages.

They are also **per-row invoked** - unlike SQL UDFs, they don't get inlined. Each row calls JavaScript separately, which is slower than pure SQL. Avoid JavaScript UDFs in hot paths on huge tables when a SQL equivalent exists.

PYTHON UDFS - FOR LIBRARIES AND ML

Student

When do I use Python UDFs?

Claude • AI Assistant

CONCEPT CARD

Python UDFs (part of Snowpark) let you write functions in Python, executed inside Snowflake's managed Python environment. They give you access to a curated set of allowed packages: **numpy, pandas, scikit-learn, xgboost, scipy**, and many more.

This is Snowflake's answer to 'I need ML/statistics in-database.'

sql — Code Example

```sql
CREATE OR REPLACE FUNCTION ml.predict_ltv(
    tenure_days INT, avg_order_value NUMBER, order_count INT
)
  RETURNS NUMBER
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('numpy')
  HANDLER = 'predict'
AS
$$
import numpy as np

def predict(tenure_days, avg_order_value, order_count):
    if tenure_days is None or avg_order_value is None or order_count is
None:
        return None
    # Toy heuristic; a real model would load a serialized artifact
    return float(np.log1p(tenure_days) * avg_order_value *
np.sqrt(order_count))
$$;

-- Usage
SELECT customer_id, ml.predict_ltv(tenure_days, avg_order_value,
order_count) AS predicted_ltv
FROM   customer_features;
```

Python UDFs are **batched via Arrow** under the hood - Snowflake sends chunks of rows to the Python process at a time, minimizing per-row overhead. For large-scale scoring, this is efficient.

For genuinely per-row Python-only logic (no vectorization possible), throughput is still slower than pure SQL.

Student

Can I load a real ML model from a stage?

Claude • AI Assistant

CONCEPT CARD

Yes - this is the standard production pattern. Upload your trained model (`.pkl` from scikit-learn, or ONNX, or a lightweight custom format) to an internal stage, then reference it in the UDF via `IMPORTS` and load it in the handler:

sql — Code Example

```sql
CREATE OR REPLACE FUNCTION ml.score_churn(features ARRAY)
  RETURNS NUMBER
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('scikit-learn', 'joblib')
  IMPORTS = ('@ml_models/churn_model_v3.pkl')
  HANDLER = 'score'
AS
$$
import sys, os, joblib
IMPORT_DIR = sys._xoptions.get('snowflake_import_directory')
MODEL = joblib.load(os.path.join(IMPORT_DIR, 'churn_model_v3.pkl'))

def score(features):
    return float(MODEL.predict_proba([features])[0][1])
$$;
```

The model file is loaded once per warehouse worker (not per row) and cached in memory. Scoring millions of rows becomes very fast because the model stays hot.

PYTHON UDTFS - ONE ROW IN, MANY ROWS OUT

Student

What if my function needs to return multiple rows per input row?

Claude • AI Assistant

CONCEPT CARD

A **Python UDTF (User-Defined Table Function)** takes a row of inputs and yields zero or more output rows. Use it for row-generating logic - explode a string into tokens, generate a time series from a start/end pair, unnest a custom-format field.

sql — Code Example

```sql
CREATE OR REPLACE FUNCTION utils.split_reference(ref STRING)
  RETURNS TABLE (position INT, token STRING)
```

```
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  HANDLER = 'RefSplitter'
AS
$$
class RefSplitter:
    def process(self, ref):
        if ref is None: return
        for pos, tok in enumerate(ref.split('-')):
            yield (pos, tok)
$$;

-- Usage with LATERAL
SELECT s.ref, t.position, t.token
FROM   staging_refs s,
  LATERAL TABLE(utils.split_reference(s.ref)) t;
```

Same idea as `LATERAL FLATTEN` for arrays, but generalized to any custom row-generation logic.

EXTERNAL FUNCTIONS

Student

What about calling an actual external API?

Claude · AI Assistant

CONCEPT CARD

An **external function** wraps an HTTPS endpoint (typically an AWS API Gateway / Lambda or Azure Function) that Snowflake can call from SQL:

sql — Code Example

```
CREATE OR REPLACE EXTERNAL FUNCTION utils.currency_convert(amount NUMBER,
from_cur STRING, to_cur STRING)
  RETURNS NUMBER
  API_INTEGRATION = api_forex_service
  AS 'https://api.acme-forex.com/v1/convert';

SELECT order_id, amount, utils.currency_convert(amount, 'USD', 'INR') AS
```

```
amount_inr
FROM   fact_orders;
```

Use external functions sparingly - each row incurs an HTTPS round trip. Snowflake batches calls behind the scenes, but latency is bounded by the external service. Best for low-volume enrichment (currency conversion, address validation) rather than hot-path scoring.

SECURE UDFS AND DATA MASKING

Student

Can UDFs hide implementation details from users?

Claude • AI Assistant

CONCEPT CARD

Add the **SECURE** keyword to a UDF (or view). Secure objects hide:

- The function body from users who can call it but not own it (their `DESCRIBE FUNCTION` shows nothing).
- Internal query optimizations - the optimizer will not evaluate underlying data unnecessarily to preserve data confidentiality.

Secure UDFs are how you enforce **row-level policies** that hide sensitive filter logic from analysts. If your UDF computes 'is this row visible to this role?', mark it SECURE so the logic itself isn't exposed.

Note: secure objects have slightly worse query performance because certain optimizations are disabled to preserve confidentiality.

ANALOGY

UDFs in Snowflake are **inline extension mechanisms**. SQL UDFs are like C preprocessor macros - substituted at parse time, zero overhead.

JavaScript and Python UDFs are like calling a subroutine - real overhead per invocation but access to a full language. External functions are like RPC calls to another service - useful but limited by network latency.

Choose the lightest mechanism that expresses your logic.

REAL WORLD APPLICATION

A retail analytics team was doing customer-tier scoring in a Python job that ran nightly, took 90 minutes, and required a whole EC2 pipeline to orchestrate. Replaced with a Python

UDF that scored inline in the fact_orders MERGE.

Scoring became continuous instead of nightly, EC2 pipeline retired, latency dropped from 24 hours to under 5 minutes. Python UDFs bring ML into your normal ETL flow without external orchestration.

⚠ WARNING

Do NOT use JavaScript or Python UDFs where a pure SQL expression would work. Per-row invocation overhead adds up on billion-row tables.

A JavaScript UDF that does `return X + Y` where X and Y are numbers is a 100x slowdown vs writing `X + Y` inline. Reserve non-SQL UDFs for logic SQL genuinely cannot express.

COMMON PITFALL / MISTAKE

Forgetting that JavaScript UDF parameter names are UPPERCASED in the body. `CREATE FUNCTION foo(bar STRING) LANGUAGE JAVASCRIPT AS $$ return bar.length; $$` returns null because `bar` is not defined inside the function - only `BAR` is.

Always use uppercase inside the JavaScript body.

EXAM TIP CARD

SnowPro Core will test: (1) SQL UDFs are single-expression and inline-optimized (fast). (2) JavaScript UDFs have UPPERCASED parameter names and are sandboxed.

(3) Python UDFs run in Snowpark's managed Python runtime with allowed packages. (4) UDTFs return tables; use with LATERAL.

(5) SECURE hides function bodies from non-owners.

PRACTICE QUIZ

You want to add GST calculation (18% of net amount) to every order row in a 5-billion-row fact table. Which UDF pattern minimizes runtime cost?

A) A JavaScript UDF that returns `NET * 0.18`

B) A Python UDF that returns `net * 0.18`

C) A SQL UDF that returns `net * 0.18`

D) An external function that calls a tax service

▼ Reveal Correct Answer

Answer: C. SQL UDFs are inline-optimized - zero runtime overhead. JS and Python UDFs pay per-row invocation cost, which on 5 billion rows becomes very significant. D adds network latency to every row. C is the same speed as writing the expression inline.

EXERCISE

For Mango Mart, write: (a) a SQL UDF that returns customer_tier ('GOLD', 'SILVER', 'BRONZE') based on lifetime spend thresholds, (b) a Python UDF that scores propensity-to-churn from a loaded sklearn model, (c) a Python UDTF that unnests a comma-separated 'preferences' string into rows.

AI DEEP DIVE PROMPT

Ask an AI: '*Compare Snowflake SQL UDF, JavaScript UDF, and Python UDF for encapsulating business logic in an ETL pipeline. Include performance benchmarks, use cases, and when each is inappropriate.*'

SUMMARY CARD

UDFs package reusable logic. **SQL UDFs** are inline-optimized (fastest); use them first.

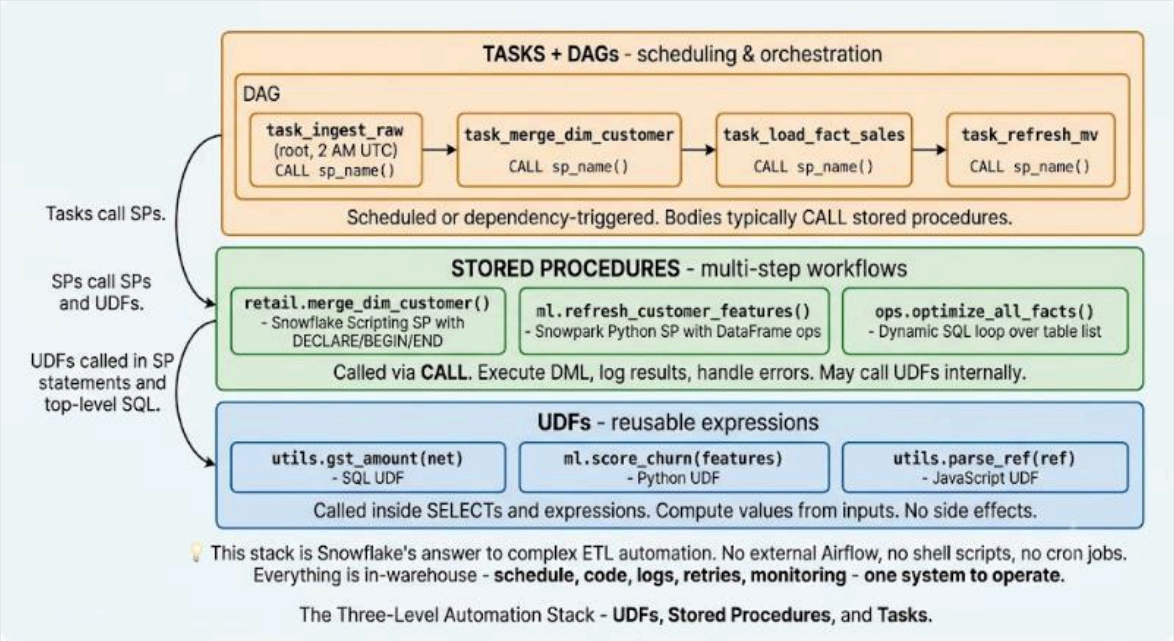
JavaScript UDFs handle per-row procedural logic SQL cannot express - sandboxed, UPPERCASED parameter names. **Python UDFs** run in Snowpark's managed Python with allowed packages (numpy, pandas, sklearn) - the standard for in-database ML scoring.

Python UDTFs yield multiple rows per input row (use with LATERAL). **External functions** call HTTPS endpoints.

SECURE hides function bodies from non-owners.

STORED PROCEDURES - MULTI-STATEMENT LOGIC AND CONTROL FLOW

Claude • AI Assistant



The Three-Level Automation Stack - UDFs, Stored Procedures, and Tasks

sql — Snowflake Scripting stored procedure - SCD Type 2 with logging and error handling

```sql
CREATE OR REPLACE PROCEDURE mango_mart.retail.merge_dim_customer()
  RETURNS VARCHAR
  LANGUAGE SQL
AS
$$
DECLARE
  started_at    TIMESTAMP_LTZ := CURRENT_TIMESTAMP();
  rows_updated  INT           := 0;
  rows_inserted INT           := 0;
BEGIN
  -- Step A: close old dim rows for customers whose data changed.
  UPDATE mango_mart.retail.dim_customer t
  SET    valid_to = :started_at, is_current = FALSE
  WHERE  t.is_current = TRUE
    AND  t.customer_id IN (
           SELECT DISTINCT raw_event:customer_id::STRING
           FROM   mango_mart.raw.strm_customer_updates
           WHERE  METADATA$ACTION = 'INSERT'
         );
  rows_updated := SQLROWCOUNT;

  -- Step B: insert new rows (both brand-new customers and new versions).
  INSERT INTO mango_mart.retail.dim_customer
    (customer_id, name, email, city, valid_from, valid_to, is_current)
```

```
  SELECT raw_event:customer_id::STRING,
         raw_event:name::STRING,
         raw_event:email::STRING,
         raw_event:city::STRING,
         :started_at,
         NULL,
         TRUE
  FROM   mango_mart.raw.strm_customer_updates
  WHERE  METADATA$ACTION = 'INSERT';
  rows_inserted := SQLROWCOUNT;

  -- Step C: audit log.
  INSERT INTO mango_mart.ops.etl_log
    (etl_name, started_at, ended_at, rows_updated, rows_inserted, status)
  VALUES ('merge_dim_customer', :started_at, CURRENT_TIMESTAMP,
          :rows_updated, :rows_inserted, 'SUCCESS');

  RETURN 'SUCCESS: ' || :rows_updated || ' updated, '
                     || :rows_inserted || ' inserted';
EXCEPTION
  WHEN OTHER THEN
    INSERT INTO mango_mart.ops.etl_log
      (etl_name, started_at, ended_at, status, error_message)
    VALUES ('merge_dim_customer', :started_at, CURRENT_TIMESTAMP,
            'FAILED', SQLERRM);
    RAISE;
END;
$$;

-- Invoke
CALL mango_mart.retail.merge_dim_customer();

-- View recent runs
SELECT etl_name, started_at, ended_at,
       DATEDIFF('SECOND', started_at, ended_at) AS runtime_sec,
       rows_updated, rows_inserted, status, error_message
FROM   mango_mart.ops.etl_log
WHERE  etl_name = 'merge_dim_customer'
ORDER  BY started_at DESC LIMIT 20;
```

sql — Snowpark Python stored procedure - DataFrame-driven feature engineering

```
CREATE OR REPLACE PROCEDURE mango_mart.ml.refresh_customer_features()
  RETURNS VARCHAR
  LANGUAGE PYTHON
```

```
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('snowflake-snowpark-python', 'pandas', 'numpy')
  HANDLER = 'refresh'
AS
$$
from snowflake.snowpark.functions import col, sum as sum_, count as count_,
max as max_, datediff, current_date, avg

def refresh(session):
    # Build feature DataFrame from fact_orders + dim_customer.
    orders = session.table('mango_mart.analytics.fact_orders')
    dim    =
session.table('mango_mart.retail.dim_customer').filter(col('is_current') ==
True)

    features = (orders
        .group_by('customer_id')
        .agg(
            sum_('net_revenue').alias('lifetime_revenue'),
            count_('order_id').alias('order_count'),
            avg('net_revenue').alias('avg_order_value'),
            max_('paid_at').alias('last_order_ts')
        )
        .with_column('days_since_last_order',
                     datediff('DAY', col('last_order_ts'), current_date()))
        .join(dim.select('customer_id', 'city', 'valid_from'),
              'customer_id', 'left')
        .with_column('tenure_days',
                     datediff('DAY', col('valid_from'), current_date()))
    )

    # Write the feature table (overwrites previous version).
    features.write.save_as_table(
        'mango_mart.ml.customer_features_current',
        mode='overwrite'
    )

    return f'Refreshed {features.count()} customer feature rows'
$$;

-- Called from a task after fact_orders is loaded.
CREATE OR REPLACE TASK mango_mart.ops.task_refresh_features
  WAREHOUSE = etl_wh
  AFTER mango_mart.ops.task_load_fact_sales
AS
  CALL mango_mart.ml.refresh_customer_features();
```

```
ALTER TASK mango_mart.ops.task_refresh_features RESUME;
```

Student

If UDFs return values, what do stored procedures do differently?

Claude • AI Assistant

A **stored procedure** is Snowflake's answer for multi-step workflows - anything you would want to write as a script with control flow (IF/ELSE, LOOP, TRY/CATCH), variables, and side-effects (INSERT, UPDATE, MERGE). UDFs compute values in expressions; stored procedures orchestrate work.

In practice: **Tasks call stored procedures**, and stored procedures do the actual heavy lifting of multi-statement ETL, admin utilities, and dynamic SQL generation.

THE THREE LANGUAGES

Student

What are my options?

Claude • AI Assistant

CONCEPT CARD

Snowflake supports stored procedures in:

Snowflake Scripting (SQL-like) - a procedural extension to SQL with IF/ELSE, FOR, WHILE, TRY/CATCH, and cursors. Best for pure SQL workflows - dynamic SQL generation, multi-step MERGE orchestration, admin tasks.

This is often the right default because it stays in the SQL world.

JavaScript - the original SP language. Full ES5+ with a Snowflake JavaScript API (`snowflake.execute()`) to run SQL. Being phased down in favor of Snowflake Scripting for pure-SQL cases and Python for data-heavy cases.

Python (Snowpark) - full Python with access to the Snowpark DataFrame API. Best for data-heavy SPs that manipulate DataFrames, orchestrate ML pipelines, or need Python libraries.

SNOWFLAKE SCRIPTING - THE STANDARD SQL SP

Student

Show me a Snowflake Scripting example.

Claude • AI Assistant

EXAMPLE CARD

A common Mango Mart use case: a stored procedure that runs the nightly SCD Type 2 MERGE, logs the result, and returns a status message.

sql — Code Example

```sql
CREATE OR REPLACE PROCEDURE retail.merge_dim_customer()
  RETURNS VARCHAR
  LANGUAGE SQL
AS
$$
DECLARE
  started_at    TIMESTAMP_LTZ := CURRENT_TIMESTAMP();
  rows_updated  INT           := 0;
  rows_inserted INT           := 0;
  status_msg    VARCHAR;
BEGIN
  -- Step A: close old rows
  UPDATE mango_mart.retail.dim_customer t
  SET    valid_to = CURRENT_TIMESTAMP, is_current = FALSE
  WHERE  t.is_current = TRUE
    AND  t.customer_id IN (
           SELECT DISTINCT raw_event:customer_id::STRING
           FROM mango_mart.raw.strm_customer_updates
           WHERE METADATA$ISUPDATE = TRUE
         );
  rows_updated := SQLROWCOUNT;

  -- Step B: insert new versions
  INSERT INTO mango_mart.retail.dim_customer (...)
  SELECT ... FROM mango_mart.raw.strm_customer_updates
  WHERE  METADATA$ACTION = 'INSERT';
  rows_inserted := SQLROWCOUNT;

  -- Step C: log result
```

```
  INSERT INTO ops.etl_log (etl_name, started_at, ended_at, rows_updated,
rows_inserted, status)
  VALUES ('merge_dim_customer', :started_at, CURRENT_TIMESTAMP,
          :rows_updated, :rows_inserted, 'SUCCESS');

  status_msg := 'SUCCESS - ' || rows_updated || ' updated, ' ||
rows_inserted || ' inserted';
  RETURN status_msg;

EXCEPTION
  WHEN OTHER THEN
    INSERT INTO ops.etl_log (etl_name, started_at, ended_at, status,
error_message)
    VALUES ('merge_dim_customer', :started_at, CURRENT_TIMESTAMP, 'FAILED',
SQLERRM);
    RAISE;
END;
$$;
```

Key elements: `DECLARE` for variables (typed), `BEGIN` / `END` for the body, `SQLROWCOUNT` for rows affected by the last DML, `EXCEPTION` blocks for error handling. `:variable_name` (colon prefix) references SQL variables inside statements.

CALL - INVOKING A PROCEDURE

Student

How do I run it?

Claude • AI Assistant

CONCEPT CARD

Use `CALL procedure_name(args);` - not `SELECT`, and not from inside another SELECT. Procedures execute as their own top-level statement:

sql — Code Example

```
CALL retail.merge_dim_customer();
-- Returns: 'SUCCESS - 234 updated, 78 inserted'

-- From a task:
CREATE OR REPLACE TASK ops.task_merge_dim_customer
```

```
  WAREHOUSE = etl_wh AFTER ops.task_ingest_raw
AS CALL retail.merge_dim_customer();
```

CONTROL FLOW IN SNOWFLAKE SCRIPTING

Student

What control-flow constructs are available?

Claude · AI Assistant

CONCEPT CARD

Snowflake Scripting supports the standard patterns:

- **IF … THEN … ELSEIF … ELSE … END IF;**
- **CASE … WHEN … THEN … ELSE … END CASE;**
- **FOR var IN start TO end DO … END FOR;**
- **FOR row_var IN cursor_var DO … END FOR;**
- **WHILE (condition) DO … END WHILE;**
- **REPEAT … UNTIL (condition) END REPEAT;**
- **BREAK** / **CONTINUE** inside loops.
- **TRY … CATCH** via **EXCEPTION** blocks.

Cursors let you iterate over query results row-by-row when needed, though set-based operations are always preferred for performance.

DYNAMIC SQL - THE KILLER USE CASE

Student

What's a case where a stored procedure really shines?

Claude · AI Assistant

EXAMPLE CARD

Dynamic SQL generation is where stored procedures shine over plain SQL. A common example: a maintenance procedure that iterates over a list of tables and applies the same operation to each:

sql — Code Example

```sql
CREATE OR REPLACE PROCEDURE ops.optimize_all_facts()
  RETURNS VARCHAR
  LANGUAGE SQL
AS
$$
DECLARE
  tbl_name VARCHAR;
  fact_cursor CURSOR FOR
    SELECT table_name FROM mango_mart.information_schema.tables
    WHERE table_schema = 'ANALYTICS' AND table_name LIKE 'FACT_%';
  total INT := 0;
BEGIN
  FOR row_var IN fact_cursor DO
    tbl_name := row_var.table_name;
    EXECUTE IMMEDIATE 'ALTER TABLE mango_mart.analytics.'
                      || tbl_name || ' RECLUSTER';
    total := total + 1;
  END FOR;
  RETURN 'Reclustered ' || total || ' fact tables';
END;
$$;

CALL ops.optimize_all_facts();
```

Pure SQL cannot do this - you cannot loop over table names and issue DDL. Stored procedures with `EXECUTE IMMEDIATE` handle it cleanly.

SNOWPARK PYTHON STORED PROCEDURES

Student

When do I use a Python stored procedure?

Claude · AI Assistant

CONCEPT CARD

Snowpark Python stored procedures run inside Snowflake and have access to a `Session` object that lets you construct Snowpark DataFrames, execute SQL, and use the full Snowflake API from Python. Use them when:

- You need Python libraries (numpy, pandas, sklearn) as part of orchestration.

- The workflow is DataFrame-heavy (transformations, filtering, joins on pandas-like objects).
- You want to unify data + ML pipelines in one procedure.

sql — Code Example

```sql
CREATE OR REPLACE PROCEDURE ml.refresh_customer_features()
  RETURNS VARCHAR
  LANGUAGE PYTHON
  RUNTIME_VERSION = '3.11'
  PACKAGES = ('snowflake-snowpark-python', 'pandas', 'numpy')
  HANDLER = 'refresh'
AS
$$
def refresh(session):
    orders = session.table('mango_mart.analytics.fact_orders')
    customers = session.table('mango_mart.retail.dim_customer')
    features = (orders
                .group_by('customer_id')
                .agg({'net_revenue': 'sum', 'order_id': 'count'})
                .join(customers, 'customer_id'))
    features.write.save_as_table('mango_mart.ml.customer_features',
                                 mode='overwrite')
    return f'Refreshed {features.count()} customer feature rows'
$$;
```

The `session` argument is injected automatically by Snowflake. You use it exactly like you would use `SparkSession` in PySpark. All computation runs inside Snowflake - no data leaves the platform.

OWNER'S RIGHTS VS CALLER'S RIGHTS

Student

Whose privileges does an SP run with?

Claude • AI Assistant

CONCEPT CARD

Two execution modes:

Owner's rights (default) - the procedure runs with the privileges of its owner. Callers can execute it even if they lack privileges on the underlying tables. This is how you build safe

interfaces that give restricted users controlled access to sensitive operations.

Caller's rights - `EXECUTE AS CALLER` - the procedure runs with the caller's own privileges. Use when the SP is meant to operate on the caller's own objects and you want the caller's role to matter.

Owner's rights is the standard for admin utilities (a procedure that grants access to newly-onboarded users runs as owner so callers don't need SECURITYADMIN). Caller's rights is standard for procedures that operate on the caller's data.

ANALOGY

UDFs are like **inline formulas** in a spreadsheet cell - compute a value and return it. Stored procedures are like **macros** that scripted a whole workflow - run several statements, branch based on results, log outcomes, handle errors.

Tasks are the **scheduler** that runs macros on a timer. UDF + SP + Task is the three-level stack Snowflake gives you for full in-warehouse automation.

REAL WORLD APPLICATION

A finance analytics team had 200+ shell scripts calling SnowSQL to run their monthly close pipeline. Migrated everything into ~15 stored procedures called by a single Task DAG.

Wall-time dropped from 6 hours to 90 minutes (SPs ran in the same session, no reconnect overhead), and the entire orchestration became visible in TASK_HISTORY instead of scattered across Jenkins logs. Stored procedures inside Snowflake beat external shell scripts on speed, observability, and maintenance cost.

WARNING

Do not do heavy row-by-row processing in a stored procedure cursor when a set-based statement would work. `FOR row IN cursor DO INSERT INTO ...` for millions of rows is a classic anti-pattern - a single `INSERT ...

SELECT` is orders of magnitude faster. Use cursors only when you genuinely need per-row logic that can't be expressed set-based.

COMMON PITFALL / MISTAKE

Confusing UDFs and stored procedures. `SELECT my_sp()` fails - stored procedures aren't callable as expressions. `CALL my_udf(...)` fails - UDFs aren't callable via CALL. Rule: UDFs are called from within SELECT/expressions; SPs are called via CALL as their own statement.

EXAM TIP CARD

SnowPro Core will test: (1) UDFs return values from expressions; SPs run workflows via CALL. (2) Snowflake Scripting is the SQL-native SP language with DECLARE/BEGIN/END.

(3) EXECUTE IMMEDIATE for dynamic SQL. (4) Owner's rights vs caller's rights - default is owner's.

(5) SPs can commit transactions; UDFs cannot.

PRACTICE QUIZ

You want to run a MERGE + UPDATE + INSERT sequence that logs to a status table and returns a summary. What tool?

A) A SQL UDF returning a status string

B) A Snowflake Scripting stored procedure with DECLARE/BEGIN/END

C) A Python UDF that calls SQL

D) A materialized view

▼ Reveal Correct Answer

Answer: B. Multi-statement DML + control flow + logging = stored procedure. A UDF cannot execute DML. C is possible via Snowpark SP but SQL is simpler here. D is unrelated.

EXERCISE

For Mango Mart, write a Snowflake Scripting stored procedure `analytics.recompute_customer_ltv()` that: (1) rebuilds a customer*ltv table from fact*orders, (2) logs before/after counts to ops.etl_log, (3) handles errors by rolling back and re-raising. Then wrap it in a task scheduled daily at 3 AM UTC.

AI DEEP DIVE PROMPT

Ask an AI: '*Compare Snowflake Stored Procedures (Snowflake Scripting, JavaScript, Snowpark Python) for a nightly ETL pipeline. Include control flow, error handling, dynamic SQL, and integration with Snowpark DataFrames.*'

SUMMARY CARD

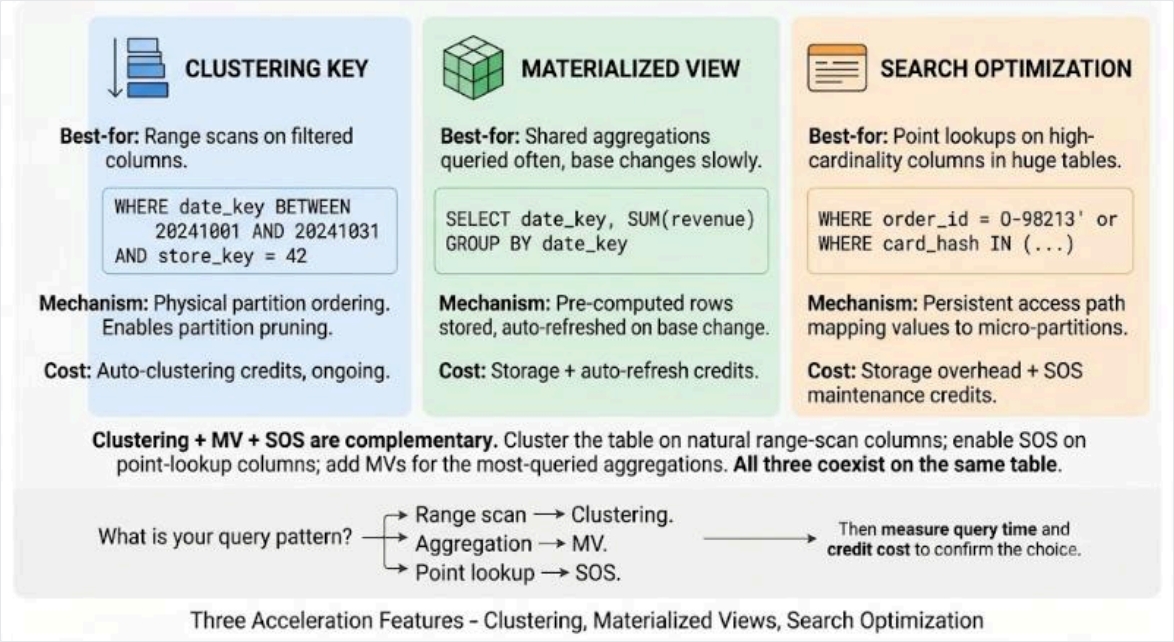
Stored Procedures encapsulate multi-statement workflows with control flow, variables, and side effects. Three languages: **Snowflake Scripting** (SQL-native, default), **JavaScript** (legacy), **Snowpark Python** (DataFrame-heavy and ML).

Called via `CALL` - not from SELECT. Use owner's rights (default) to give restricted users safe access.

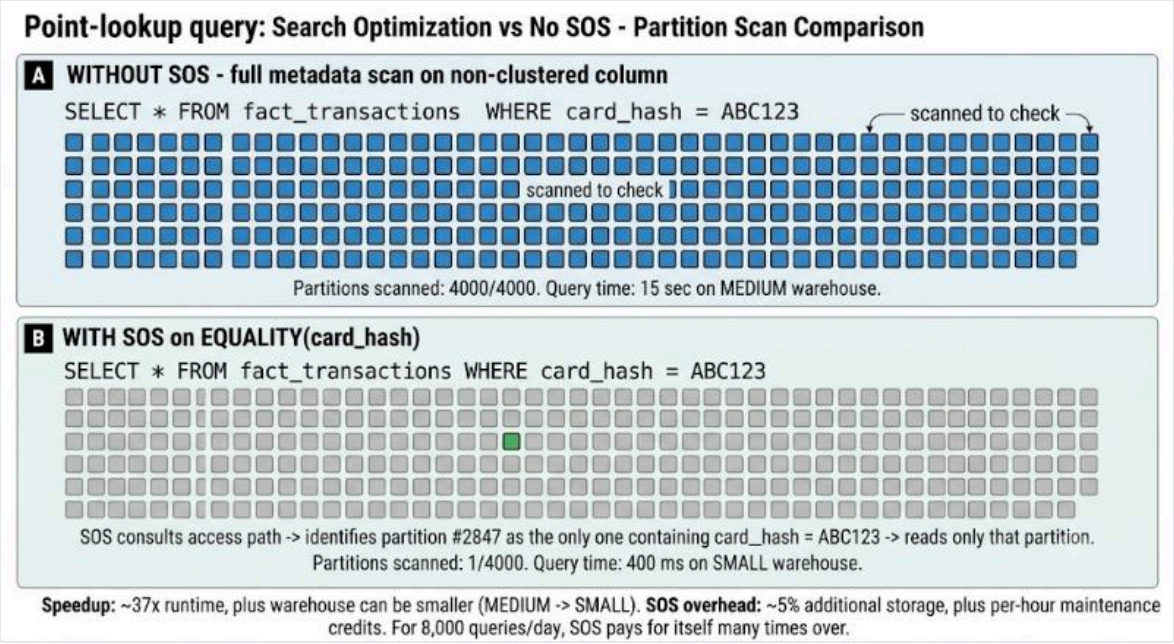
> Combined with Tasks and DAGs, SPs are the standard way to build in-warehouse orchestration without external tools.

SEARCH OPTIMIZATION SERVICE - POINT LOOKUPS AT SCALE

Claude • AI Assistant



Three Acceleration Features - Clustering, Materialized Views, Search Optimization



Search Optimization vs No SOS - Partition Scan Comparison

sql — Search Optimization - enable, use, monitor, and reason about cost

```sql
-- 1. Enable SOS on specific columns and methods.
-- EQUALITY is the default; other methods must be named.
ALTER TABLE mango_mart.analytics.fact_sales_transaction
  ADD SEARCH OPTIMIZATION
  ON EQUALITY(order_id, customer_id),
     SUBSTRING(product_ref);

-- 2. Check SOS is built (async - may take minutes to hours for large
tables).
SHOW TABLES LIKE 'FACT_SALES_TRANSACTION' IN SCHEMA mango_mart.analytics;
-- Look for search_optimization column = 'ON' with a progress %

-- More detailed status:
SELECT
SYSTEM$SEARCH_OPTIMIZATION_INFORMATION('mango_mart.analytics.fact_sales_trans
action');

-- 3. Run a point-lookup query - benefits from SOS automatically.
SELECT * FROM mango_mart.analytics.fact_sales_transaction
WHERE  order_id = 'O-98213-2024';
-- Check Query Profile - look for 'Search Optimization Access' node.

-- 4. Monitor SOS credit consumption per table (last 30 days).
SELECT
  table_name,
  ROUND(SUM(credits_used), 2)             AS sos_credits_30d,
  ROUND(SUM(credits_used) * 3.0, 2)       AS approx_usd_30d
FROM   snowflake.account_usage.search_optimization_history
WHERE  start_time >= DATEADD('DAY', -30, CURRENT_TIMESTAMP())
GROUP  BY table_name
ORDER  BY sos_credits_30d DESC;

-- 5. Storage overhead from SOS access path.
SELECT table_name,
       ROUND(active_bytes                / POWER(1024,3), 2) AS active_gb,
       ROUND(search_optimization_bytes   / POWER(1024,3), 2) AS sos_gb,
       ROUND(100.0 * search_optimization_bytes
                / NULLIF(active_bytes, 0), 1)                AS sos_pct
FROM   snowflake.account_usage.table_storage_metrics
WHERE  table_name = 'FACT_SALES_TRANSACTION' AND deleted = FALSE;

-- 6. Disable if maintenance cost > savings.
```

```
-- ALTER TABLE mango_mart.analytics.fact_sales_transaction
--   DROP SEARCH OPTIMIZATION;
```

Student

Chapter 3 covered clustering keys for range scans and materialized views for aggregations. But what about the opposite pattern - looking up a single row by a high-cardinality identifier in a huge table? Like 'find order O-98213' in a 10-billion-row fact table.

Claude • AI Assistant

That's exactly what the **Search Optimization Service (SOS)** is for. SOS is an opt-in Snowflake feature that builds and maintains a persistent access path optimized for **point lookups** and **selective predicates** on huge tables.

Where clustering keys help scan a range of physically-adjacent rows fast, SOS helps find a specific needle (or a few needles) in the haystack. It's Snowflake's answer to the 'OLTP-like query on an OLAP table' pattern.

WHAT SOS ACTUALLY IS

Student

Is it like an index?

Claude • AI Assistant

CONCEPT CARD

Conceptually similar, but not the same as traditional B-tree or hash indexes. **SOS builds a per-table search access path** that lets Snowflake:

1. Quickly identify which micro-partitions contain rows matching a highly-selective predicate.
2. Efficiently retrieve just those rows without scanning many partitions.

Under the hood SOS stores a data structure (a kind of specialized secondary access path) that maps values to the micro-partitions containing them. When you query with a selective

WHERE clause, Snowflake consults SOS first, prunes down to only the relevant partitions, and reads only those.

On a 10-billion-row table where the pattern-of-interest hits just 50 rows, SOS can find and return them in **milliseconds**.

THE FOUR SOS METHODS

Student

What kinds of predicates does SOS accelerate?

Claude • AI Assistant

CONCEPT CARD

SOS has **four methods** you can enable per column:

EQUALITY (default) - point lookups: `WHERE order_id = 'O-98213'` , `WHERE customer_id IN (12, 47, 89)` . The most common use.

SUBSTRING - `LIKE` , `ILIKE` , `RLIKE` predicates with wildcards: `WHERE order_id LIKE 'O-98%'` . Enables efficient substring matching without full scans.

GEO - geospatial predicates on GEOGRAPHY columns: `WHERE ST_CONTAINS(region_polygon, ST_MAKEPOINT(lon, lat))` .

VARIANT - selective predicates on paths inside VARIANT columns: `WHERE event_data:user_id::STRING = 'u-12345'` . Very useful for semi-structured event data.

You enable methods explicitly: `ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id), SUBSTRING(product_ref)` .

WHEN TO USE SOS

Student

When is SOS the right choice vs a clustering key or an MV?

Claude • AI Assistant

CONCEPT CARD

SOS fits when all of these are true:

- **The table is very large** (100 GB+ / 1 billion+ rows) - pruning matters at this scale.
- **Queries do point or highly-selective lookups** returning a small number of rows.
- **The predicate column is NOT the clustering key** - if it were, pruning would already work.
- **The lookup pattern is frequent** enough to justify the ongoing maintenance cost.

SOS is the wrong tool when:

- You're doing range scans (clustering keys are better).
- You're doing wide aggregations (MVs are better).
- Queries return large row percentages (SOS doesn't help; full scan is honest).
- Table is small (natural pruning handles it).

THE COST MODEL

Student

What does SOS cost?

Claude • AI Assistant

⚠ **WARNING**

SOS has two ongoing costs:

Storage - the search access path adds bytes to your table's storage. Roughly 1-30% overhead depending on cardinality and method. Check `TABLE_STORAGE_METRICS.SEARCH_OPTIMIZATION_BYTES` .

Serverless maintenance credits - as new data lands, SOS updates its access path in the background. This runs serverlessly (no user warehouse) and bills per SOS credit hour. Check `AUTOMATIC_CLUSTERING_HISTORY` and `SEARCH_OPTIMIZATION_HISTORY` .

For a stable table with predictable point-lookup patterns, SOS often pays off - the query-time cost savings far exceed maintenance credits. For a high-churn table where every load invalidates most access-path entries, SOS may cost more than it saves. Measure before committing.

THE ECONOMIC DECISION

Student

How do I decide?

Claude • AI Assistant

EXAMPLE CARD

The economic calculation:

Cost without SOS: every point-lookup query scans many partitions. Say 100 lookups/day at 5 seconds on a MEDIUM warehouse = 500 seconds/day = ~ 0.15/day = 4.50/month.

Cost with SOS: point lookups scan ~1 partition each = 100 milliseconds = ~ 0.01/dayquerycost, plus 50-200/month SOS maintenance (depends on churn).

For 100 lookups/day, SOS is NET COST - the maintenance exceeds the savings. But for 10,000+ lookups/day (typical for a customer-service tool querying a fact table), SOS is a NET WIN by an order of magnitude.

Rule of thumb: enable SOS when you have thousands of point-lookup queries per day on a large table, and the alternative (clustering the column, or running lookups against a smaller derived table) is worse.

COMBINING FEATURES

Student

Can I combine SOS with clustering keys?

Claude • AI Assistant

Yes. A common pattern for `fact_sales_transaction` :

- **CLUSTER BY (date_key, store_key)** for daily/store-level range scans (BI dashboards).
- **ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id)** for point lookups (customer-service tool, returns lookup).

The two mechanisms don't conflict - clustering handles range scans, SOS handles points. Both maintain themselves automatically. Storage overhead is the sum of both features, so watch total storage.

MONITORING SOS

Student

How do I see if SOS is helping?

Claude • AI Assistant

CONCEPT CARD

Two checks:

Runtime - Query Profile shows a special node 'Search Optimization Access' when SOS is used. If your point-lookup queries don't show this node, SOS isn't helping - maybe your predicate isn't SOS-eligible, or SOS isn't yet built for the column.

Cost - `SNOWFLAKE.ACCOUNT_USAGE.SEARCH_OPTIMIZATION_HISTORY` shows credit consumption per table per day. Compare to the query-time savings you would have paid without SOS. If maintenance credits > savings, disable SOS.

ANALOGY

Search Optimization Service is like adding a **detailed index at the back of an encyclopedia**. Without the index, finding a specific entry (say, 'Snowpipe') means scanning the whole book.

With the index, you jump straight to the right page. The index takes some pages of its own (storage cost) and has to be updated whenever new entries are added (maintenance cost) - so it's worth it only when you look up specific entries often.

Clustering keys are more like putting the encyclopedia entries in alphabetical order (range scan help); the index is the point-lookup accelerator on top.

REAL WORLD APPLICATION

A customer-service application at a payments company needed to look up 'give me all transactions for card ending 4417 in the last 30 days' hundreds of times per hour from a 40-billion-row transaction table. Without SOS, each lookup took ~15 seconds on an XL warehouse.

With SOS on EQUALITY(card*last*four), lookups dropped to under 500 ms on a SMALL warehouse. Monthly compute savings: about $18,000.

SOS maintenance cost: 600/month.Netwin : 17,400/month for a single feature toggle.

WARNING

SOS does NOT help when your WHERE clause disables pruning through non-sargable predicates. `WHERE UPPER(order_id) = 'O-98213'` blocks SOS just like it blocks clustering. Sargability is a prerequisite for every acceleration feature.

COMMON PITFALL / MISTAKE

Enabling SOS on a low-cardinality column like `status` (3 possible values). Every predicate matches billions of rows - SOS provides no partition-elimination benefit and costs storage + maintenance.

Enable SOS only on high-cardinality columns where selective predicates return a tiny fraction of rows.

EXAM TIP CARD

SnowPro Core will test: (1) SOS accelerates point lookups on huge tables; complements clustering (range scans) and MVs (aggregations). (2) Four methods: EQUALITY, SUBSTRING, GEO, VARIANT.

(3) Serverless maintenance - no user warehouse. (4) Not free - storage + maintenance credits ongoing.

(5) Best for high-frequency selective queries.

PRACTICE QUIZ

You have a 5-billion-row `fact_transactions` table clustered by `(transaction_date, store_id)`. Your customer-service app runs 5,000 queries/day of the form `SELECT * FROM fact_transactions WHERE card_number_hash = 'x...'`. Best acceleration?

A) Add clustering key on card*number*hash

B) Enable Search Optimization on EQUALITY(card*number*hash)

C) Create a materialized view grouped by card*number*hash

D) Increase warehouse size

▼ Reveal Correct Answer

Answer: B. High-cardinality point lookup on a non-clustered column, high query frequency = SOS. A would break the existing beneficial clustering. C doesn't help point lookups. D throws money at the problem without addressing pruning.

EXERCISE

For Mango Mart's fact*sales*transaction (4 billion rows, clustered by store*key and date*key), the customer-service team runs 8,000 queries/day looking up specific orders by order_id (LIKE 'O-98%'). (a) Which SOS method to enable?

(b) What is the expected cost/benefit? (c) How would you verify SOS is actually being used in the query plan?

AI DEEP DIVE PROMPT

Ask an AI: '*Explain how Snowflake's Search Optimization Service works in relation to clustering keys and micro-partition pruning. Include a decision matrix for when to use SOS vs clustering keys vs materialized views for different query patterns.*'

SUMMARY CARD

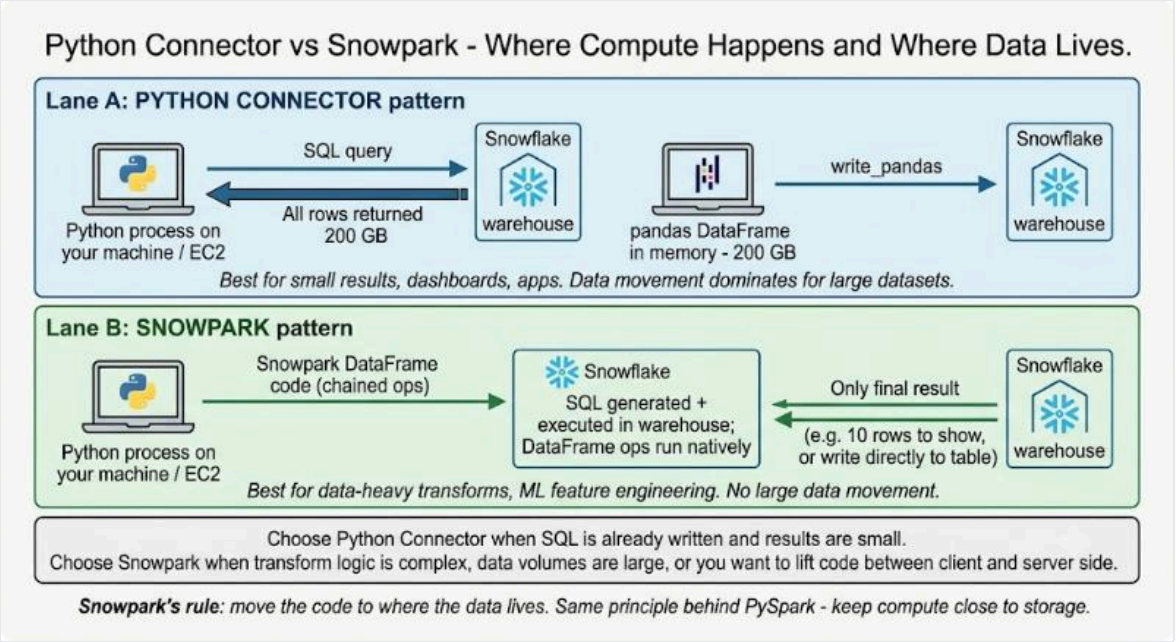
Search Optimization Service (SOS) builds and maintains a persistent access path for point lookups and highly-selective predicates on huge tables. Four methods: **EQUALITY** (default point lookups), **SUBSTRING** (LIKE with wildcards), **GEO** (geospatial), **VARIANT** (paths in semi-structured columns).

Serverless maintenance with per-hour credit cost. Complements clustering (range scans) and MVs (aggregations); use for high-frequency selective queries on very large tables.

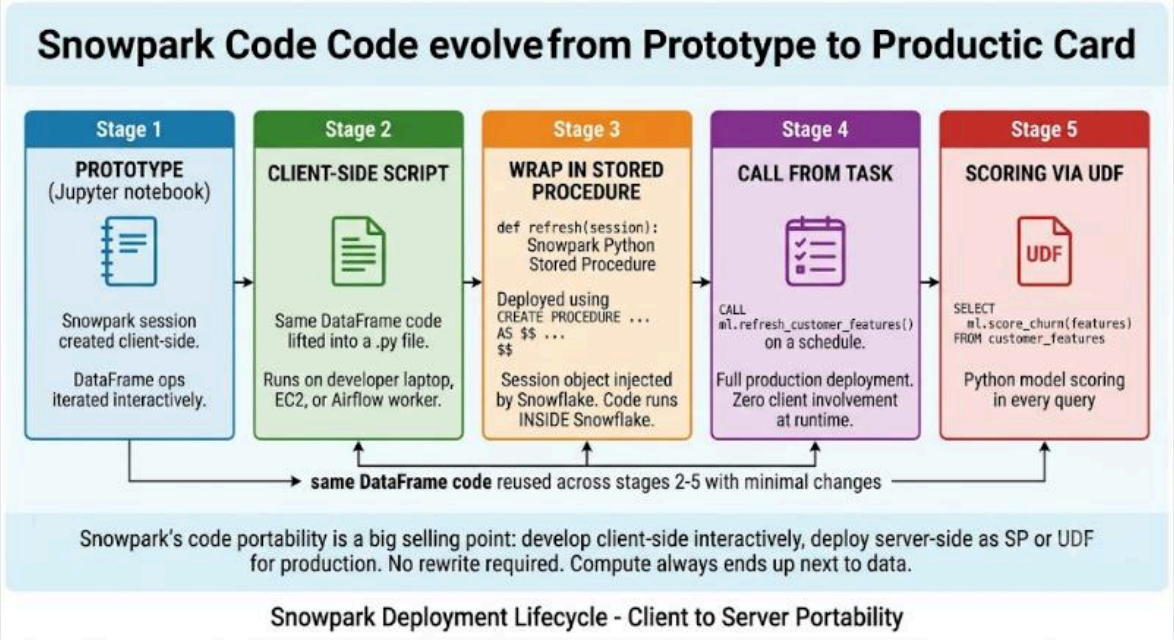
Not the answer for low-frequency lookups or low-cardinality columns.

PYTHON CONNECTOR AND SNOWPARK - PROGRAMMATIC ACCESS TO SNOWFLAKE

Claude • AI Assistant



Python Connector vs Snowpark - Where Compute Happens and Where Data Lives



Snowpark Deployment Lifecycle - Client to Server Portability

python — Python Connector - DB-API 2.0 basics + pandas integration

```python
import snowflake.connector
from snowflake.connector.pandas_tools import write_pandas
import pandas as pd

# --- Connect ---
# Production: use key-pair authentication, not password.
conn = snowflake.connector.connect(
    user='priya',
    private_key_file='/path/to/rsa_key.p8',
    private_key_file_pwd='***',
    account='xy12345.ap-south-1',
    warehouse='ANALYST_WH',
    database='MANGO_MART',
    schema='ANALYTICS',
    role='ANALYST',
)

cur = conn.cursor()

# --- Execute a parameterized query (never string-interpolate user input) ---
cur.execute(
    'SELECT store_key, SUM(net_revenue) AS total '
    'FROM fact_sales_transaction '
    'WHERE date_key BETWEEN %s AND %s '
    'GROUP BY store_key ORDER BY total DESC LIMIT 10',
    (20241101, 20241130)
)
```

```
for store_key, total in cur.fetchall():
    print(f'Store {store_key}: {total:,.2f}')

# --- Fetch as pandas DataFrame (good for small-to-medium result sets) ---
cur.execute('SELECT * FROM dim_customer WHERE is_current = TRUE LIMIT 10000')
df = cur.fetch_pandas_all()
print(df.head())

# --- Bulk-write a DataFrame back to Snowflake (uses internal stage) ---
scores_df = pd.DataFrame({
    'customer_id': ['C-1', 'C-2', 'C-3'],
    'churn_score': [0.12, 0.87, 0.45],
})
success, nchunks, nrows, _ = write_pandas(
    conn, scores_df,
    table_name='CUSTOMER_CHURN_SCORES',
    database='MANGO_MART', schema='ML',
    auto_create_table=True,     # creates the table if it does not exist
    overwrite=False
)
print(f'Wrote {nrows} rows in {nchunks} chunk(s), success={success}')

cur.close()
conn.close()
```

python — Snowpark Python - DataFrame API pushing compute into Snowflake

```
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, sum as sum_, count as count_,
avg, max as max_, datediff, current_date

# --- Session (client-side; compute stays in Snowflake) ---
session = Session.builder.configs({
    'user': 'priya',
    'private_key_file': '/path/to/rsa_key.p8',
    'private_key_file_pwd': '***',
    'account': 'xy12345.ap-south-1',
    'warehouse': 'ANALYST_WH',
    'database': 'MANGO_MART',
    'schema': 'ANALYTICS',
    'role': 'ANALYST',
}).create()

# --- DataFrame ops - lazy; only run when an action is called ---
orders = session.table('fact_sales_transaction')
```

```
customers =
session.table('mango_mart.retail.dim_customer').filter(col('is_current') ==
True)

customer_features = (
    orders
      .filter(col('date_key') >= 20240101)   # cut down at source
      .group_by('customer_id')
      .agg(
          sum_('net_revenue').alias('lifetime_revenue'),
          count_('order_id').alias('order_count'),
          avg('net_revenue').alias('avg_order_value'),
          max_('paid_at').alias('last_order_ts'),
      )
      .with_column('days_since_last_order',
                   datediff('DAY', col('last_order_ts'), current_date()))
      .join(customers.select('customer_id', 'city'), 'customer_id', 'left')
)

# --- Action 1: show top 10 rows in the driver (small result travels back) --
-
customer_features.sort(col('lifetime_revenue').desc()).limit(10).show()

# --- Action 2: write back to Snowflake as a table (nothing travels to
client) ---
customer_features.write.save_as_table(
    'mango_mart.ml.customer_features_current',
    mode='overwrite'
)

# --- Action 3: collect to pandas ONLY for small results ---
top10 =
customer_features.sort(col('lifetime_revenue').desc()).limit(10).to_pandas()
print(top10)

session.close()
```

Student

Now the last major piece - how do I connect to Snowflake from Python for building applications or ML workflows?

Claude • AI Assistant

Snowflake has two Python interfaces you should know. The **snowflake-connector-python** library is the low-level DB-API 2.0 driver - the standard way to run SQL from Python, get back rows, and integrate Snowflake into Python applications, dashboards, and pipelines.

Snowpark for Python is the higher-level DataFrame API that lets you write Python code that runs natively inside Snowflake, similar to PySpark. Combined with Python UDFs (Chapter 4.3) and Python stored procedures (4.4), Snowpark is Snowflake's answer to 'bring your Python code close to the data instead of moving data to your code.'

THE PYTHON CONNECTOR - FUNDAMENTALS

Student

Show me the basics.

Claude • AI Assistant

EXAMPLE CARD

The Python Connector uses standard DB-API 2.0 patterns:

python — Code Example

```python
import snowflake.connector

conn = snowflake.connector.connect(
    user='priya',
    password='***',                # or use SSO/keypair
    account='xy12345.ap-south-1',
    warehouse='ANALYST_WH',
    database='MANGO_MART',
    schema='ANALYTICS',
    role='ANALYST',
)

cur = conn.cursor()
cur.execute("SELECT store_key, SUM(net_revenue) FROM fact_sales_transaction
"
            "WHERE date_key >= 20241101 GROUP BY store_key ORDER BY 2 DESC
LIMIT 10")
for row in cur.fetchall():
    print(row)
```

```
cur.close()
conn.close()
```

Same patterns as psycopg2 for Postgres or mysql-connector for MySQL. Straightforward.

Student

What about parameterized queries?

Claude · AI Assistant

⚠ **WARNING**

Always use parameterized queries (`%s` placeholders with an args tuple), never string interpolation - this prevents SQL injection:

python — Code Example

```python
# GOOD - parameterized
cur.execute(
    "SELECT * FROM dim_customer WHERE customer_id = %s",
    (customer_id,)
)

# BAD - vulnerable to injection
cur.execute(f"SELECT * FROM dim_customer WHERE customer_id =
'{customer_id}'")
```

READING AND WRITING DATAFRAMES

Student

Can I use pandas with the connector?

Claude · AI Assistant

CONCEPT CARD

Yes - the connector exposes helpers for pandas:

python — Code Example

```
# Read query results into a DataFrame
df = cur.fetch_pandas_all()

# Or with pandas.read_sql
import pandas as pd
df = pd.read_sql('SELECT * FROM dim_customer LIMIT 1000', conn)

# Write a DataFrame back to Snowflake (bulk upload via internal stage)
from snowflake.connector.pandas_tools import write_pandas
success, nchunks, nrows, _ = write_pandas(
    conn, df, table_name='CUSTOMER_SCORES', database='MANGO_MART',
schema='ML'
)
```

`write_pandas` is efficient - it uses an internal stage under the hood and does a bulk COPY INTO. For datasets under a few million rows this is the right tool.

Student

What about larger datasets?

Claude • AI Assistant

⚠ WARNING

For **very large** datasets (10 million+ rows or gigabytes), pulling into pandas or pushing from pandas is inefficient - you're moving data over the network. Instead, do the transformation in Snowflake with SQL or Snowpark DataFrames, and only pull the result.

Move the code, not the data - the same principle that drove Snowpark's design.

AUTHENTICATION OPTIONS

Student

What are the authentication choices?

Claude • AI Assistant

CONCEPT CARD

Beyond password:

Key-pair authentication - generate an RSA keypair; upload public key to your user; sign JWTs with private key. Standard for service accounts and CI/CD pipelines. Never expires; can be rotated.

SSO / OAuth - integrate with Okta, Azure AD, or another identity provider. Users log in via browser; Snowflake receives an OAuth token. Standard for interactive Python notebooks in enterprise environments.

MFA (Duo) - password + push notification. Interactive use only; not for automation.

External browser (SSO) - Snowflake opens the browser to your IdP; the notebook waits for authentication.

For production automation, **key-pair authentication is the standard** - no passwords in scripts, no interactive prompts, rotates cleanly.

SNOWPARK PYTHON - DATAFRAMES IN THE DATABASE

Student

Now show me Snowpark.

Claude • AI Assistant

CONCEPT CARD

Snowpark Python is a DataFrame API modeled on PySpark. You write Python code with `DataFrame.filter().group_by().agg()` style transformations, and Snowpark translates them to SQL that runs entirely inside Snowflake:

python — Code Example

```python
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, sum as sum_, count

session = Session.builder.configs({
    'user': 'priya', 'password': '***',
    'account': 'xy12345.ap-south-1',
    'warehouse': 'ANALYST_WH', 'database': 'MANGO_MART', 'schema':
'ANALYTICS'
}).create()

orders = session.table('fact_sales_transaction')
result = (orders
  .filter(col('date_key') >= 20241101)
```

```
    .group_by('store_key')
    .agg(sum_('net_revenue').alias('total'), count('*').alias('txn_count'))
    .sort(col('total').desc())
    .limit(10))

result.show()   # Executes SQL in Snowflake, returns just 10 rows to client.
```

Key insight: the `.filter().group_by().agg()` chain is **lazy** - nothing runs until `.show()`, `.collect()`, `.to_pandas()`, or `.save_as_table()` is called. When it does run, Snowpark generates a single SQL query that executes in the warehouse.

Only the result (10 rows in this case) travels back to your Python process.

SNOWPARK VS PYTHON CONNECTOR

Student

When do I use which?

Claude • AI Assistant

CONCEPT CARD

Python Connector is right when:

- You already know the SQL and want to execute it directly.
- You are building a dashboard or app that queries specific rows.
- You need DB-API 2.0 compatibility with tooling.

Snowpark is right when:

- Your transformation logic is complex and reads better as chained DataFrame ops than as CTEs.
- You want to reuse the same code as UDFs, stored procedures, and client-side DataFrames.
- You are moving code from PySpark - Snowpark feels almost identical.
- You want Snowflake to plan and execute the whole pipeline as one query rather than round-tripping.

CLIENT-SIDE VS SERVER-SIDE SNOWPARK

Student

You mentioned Snowpark runs in Snowflake for UDFs and SPs. What about the client-side DataFrame code?

Claude • AI Assistant

Client-side Snowpark generates SQL and pushes it to Snowflake to execute - the client is just orchestrating; no data crunching happens locally. Server-side Snowpark (inside a Python UDF or SP) runs actual Python code inside Snowflake's Python runtime.

Both patterns push compute to Snowflake; the difference is where the Python interpreter runs.

The magic is that the same DataFrame code can be lifted from client to server-side almost unchanged - you develop and test client-side, then wrap in a stored procedure for production deployment. This code portability is a big Snowpark selling point.

ASYNC AND MULTI-STATEMENT PATTERNS

Student

What about running multiple queries or waiting for long-running ones?

Claude • AI Assistant

CONCEPT CARD

The connector supports:

Async queries - `cur.execute_async(...)` returns immediately; `cur.get_results_from_sfqid(qid)` retrieves the results later. Useful for long-running ETL - kick off in one process, check status in another.

Multi-statement blocks - execute several SQL statements as one call by setting `MULTI_STATEMENT_COUNT` in session parameters, then passing multiple semicolon-separated statements to `cur.execute()` .

Query IDs - every query gets a UUID; you can retrieve results, cancel, or check status by ID. Powerful for building resumable pipelines.

ANALOGY

Snowflake's Python interfaces span the whole spectrum from **thin SQL client** (Python Connector - like psycopg2) to **DataFrame engine that ships code to data** (Snowpark -

like PySpark). Same principle as PySpark's evolution: move heavy compute to where the data lives instead of pulling data over the network to compute.

REAL WORLD APPLICATION

A media analytics team had a 400-line pandas script that pulled 200 GB from Snowflake each night, ran feature engineering on a big EC2 box, and wrote results back. Total wall time: 3 hours (data movement dominated).

Rewrote in Snowpark DataFrames - the same logic, but all compute stayed inside Snowflake, only the final feature table was written back. New wall time: 12 minutes.

They also retired the EC2 box entirely. Snowpark won on latency AND infrastructure cost by keeping data at rest.

WARNING

`pd.read_sql('SELECT * FROM huge_table', conn)` on a 100-million-row table will attempt to pull all 100 million rows through your Python process's memory. It will probably crash and definitely be slow.

Add `LIMIT` clauses or write predicates that reduce the row count before pulling to pandas. Better: use Snowpark or run the aggregation in SQL and pull only the small result.

COMMON PITFALL / MISTAKE

Building a Python loop that runs one Snowflake query per iteration for a large collection. Each query has ~100-500ms setup overhead - looping 1,000 times = many minutes of pure overhead.

Batch into fewer queries with `IN (...)` lists or JOIN against a temporary table you populated in bulk.

EXAM TIP CARD

SnowPro Core will test: (1) Python Connector implements DB-API 2.0 - cursor/execute/fetch. (2) `write_pandas` uses an internal stage for bulk upload.

(3) Key-pair authentication is the production standard for automation. (4) Snowpark DataFrames are lazy; execution triggers on show/collect/save.

(5) Snowpark computation runs inside Snowflake, not on the client.

PRACTICE QUIZ

You need to run a Python script that reads 1 billion rows, does feature engineering, and writes 50 million rows back. What's the best architecture?

A) Python Connector with pd.read*sql, transform in pandas, write*pandas

B) Python Connector executing pure SQL for the transform

C) Snowpark DataFrame API with client-side session

D) Copy the data to S3, transform with EMR, load back

▼ Reveal Correct Answer

Answer: C. Snowpark keeps compute in Snowflake - only orchestrates from client. A moves data to client (fails at this scale). B is fine but Snowpark reads more naturally. D adds infrastructure and network hops.

EXERCISE

For Mango Mart, build a Snowpark Python script that: (a) reads fact*sales*transaction, (b) computes customer LTV features (total revenue, order count, avg order, recency), (c) joins with dim*customer, (d) writes to a new customer*features table. Time it and compare to the equivalent SQL query.

AI DEEP DIVE PROMPT

Ask an AI: '*Show me how to migrate a PySpark data pipeline to Snowpark Python for Snowflake. Include DataFrame operations, UDF integration, and stored-procedure deployment. Highlight where Snowpark differs from PySpark.*'

SUMMARY CARD

snowflake-connector-python is the DB-API 2.0 driver - execute SQL, fetch rows, integrate into apps. Use `write_pandas` for bulk uploads.

Snowpark Python is a PySpark-style DataFrame API where operations translate to SQL running inside Snowflake - lazy execution, compute stays with data. Client-side Snowpark orchestrates; server-side (UDFs, SPs) runs Python inside Snowflake.

Key-pair authentication is the production standard for automation. Move code to data, not data to code.

CHAPTER REVIEW & SUMMARY

Claude • AI Assistant

```
sql — Chapter 4 in a single snapshot - the six production patterns in one runnable
script
```

```
-- 4.1 Stream for CDC on Snowpipe landing table
CREATE OR REPLACE STREAM mango_mart.raw.strm_new_orders
  ON TABLE mango_mart.raw.raw_orders_stream
  APPEND_ONLY = TRUE;

-- 4.4 Stored procedure that consumes the stream via MERGE
CREATE OR REPLACE PROCEDURE mango_mart.analytics.merge_new_orders()
  RETURNS VARCHAR LANGUAGE SQL AS
$$
BEGIN
  MERGE INTO mango_mart.analytics.fact_orders t
  USING (SELECT raw_event:order_id::STRING AS order_id,
                raw_event:customer.id::INT AS customer_id
         FROM mango_mart.raw.strm_new_orders
         WHERE METADATA$ACTION = 'INSERT') s
  ON  t.order_id = s.order_id
  WHEN NOT MATCHED THEN INSERT (order_id, customer_id)
                        VALUES (s.order_id, s.customer_id);
  RETURN 'MERGED ' || SQLROWCOUNT || ' rows';
END;
$$;

-- 4.2 Task that calls the SP every 5 min ONLY if the stream has data
CREATE OR REPLACE TASK mango_mart.ops.task_merge_orders
  WAREHOUSE = etl_wh
  SCHEDULE = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('mango_mart.raw.strm_new_orders')
AS
  CALL mango_mart.analytics.merge_new_orders();

ALTER TASK mango_mart.ops.task_merge_orders RESUME;

-- 4.3 SQL UDF - inline-optimized business logic
CREATE OR REPLACE FUNCTION mango_mart.utils.gst_amount(net NUMBER)
  RETURNS NUMBER LANGUAGE SQL AS $$ net * 0.18 $$;

-- 4.5 Search Optimization for customer-service point lookups
ALTER TABLE mango_mart.analytics.fact_sales_transaction
  ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_id);

-- 4.6 Snowpark from Python (client) - executes in Snowflake:
-- from snowflake.snowpark import Session
-- session = Session.builder.configs({...}).create()
-- session.table('fact_orders').filter(col('date_key') >= 20241101).show()
```

CHAPTER SUMMARY

Chapter 4 turned Snowflake's advanced features into complete, production-shaped patterns. Every mechanism you learned in Chapter 3 (Snowpipe, Time Travel, cloning, clustering, VARIANT, RBAC) now composes into workflows engineers actually build in production.

THE SIX PRODUCTION PATTERNS

1. Streams (4.1). Change Data Capture inside Snowflake without external tools.

Streams track INSERT/UPDATE/DELETE via METADATA$ACTION and METADATA$ISUPDATE, advance their offset only on successful DML consumption, and depend on source Time Travel. APPEND_ONLY streams for high-volume inserts.

Multiple streams on the same source enable independent downstream consumers. Foundation for SCD Type 2 pipelines.

2. Tasks & DAGs (4.2). Snowflake-native scheduling and orchestration.

Standalone tasks (SCHEDULE) or dependency-triggered (AFTER) into arbitrary DAGs. Created SUSPENDED - explicit RESUME.

Serverless option for bursty workloads. WHEN SYSTEM$STREAM*HAS*DATA skips empty runs without warehouse activation.

Auto-suspend after 5 consecutive failures. Eliminates external orchestrators for Snowflake-internal pipelines.

3. UDFs (4.3). Reusable functions in SQL (inline-optimized, fastest), JavaScript (per-row procedural work), Python (numpy/pandas/sklearn access), and Python UDTFs (row-generating).

Python UDFs with IMPORTS load ML models from stages; scoring happens inline in SQL. SECURE UDFs hide implementation.

4. Stored Procedures (4.4). Multi-statement workflows with control flow, variables, error handling, and side effects.

Three languages: Snowflake Scripting (SQL-native), JavaScript (legacy), Snowpark Python (DataFrame-heavy). Called via CALL, not from SELECT.

Owner's rights (default) enables safe delegation. Task DAGs typically call stored procedures for actual work.

5. Search Optimization Service (4.5). Persistent access path for point lookups and highly-selective predicates on huge tables.

Four methods: EQUALITY, SUBSTRING, GEO, VARIANT. Complements clustering (range scans) and MVs (aggregations).

Serverless maintenance with ongoing cost. Best for high-frequency lookups on billion-row tables.

6. Python Connector & Snowpark (4.6). Python Connector for DB-API 2.0 SQL execution with pandas integration.

Snowpark for PySpark-style DataFrame API where compute stays in Snowflake. Key-pair authentication for production automation.

Same DataFrame code portable between client-side prototyping and server-side deployment (UDFs, SPs). Move code to data, not data to code.

ONE-SENTENCE CHAPTER DISTILLATION

Snowflake real-world patterns - **Streams + Tasks** for continuous CDC and orchestration, **UDFs + Stored Procedures** for reusable in-warehouse logic, **Search Optimization** for point-lookup performance at scale, and **Python Connector + Snowpark** for programmatic access - compose Snowflake's foundational and advanced features into full production data platforms with no external tooling required.

END-OF-CHAPTER CAPSTONE

> **EXERCISE**
>
> Build the complete Mango Mart real-time customer analytics pipeline end-to-end. Timebox: 4 hours.
>
> 1. **Snowpipe** landing customer update events from S3 to `raw.raw_customer_updates` (VARIANT).
> 2. **Stream** `strm_customer_updates` on that table capturing all changes.
> 3. **Stored Procedure** `retail.merge_dim_customer()` implementing SCD Type 2 (from 4.4) with error handling and etl_log entries.
> 4. **Task DAG**: root task `task_snowpipe_health` scheduled every 5 minutes, child `task_merge_dim_customer` AFTER it that calls the SP.
> 5. **Python UDF** `ml.score_churn` loading a joblib model from a stage, scoring customers on demand.
> 6. **Snowpark Python SP** `ml.refresh_customer_features` building a customer feature table from fact*orders* + *dim*customer, scheduled via another task.
> 7. **Search Optimization** on the `fact_sales_transaction` for customer-service lookups by order_id.
> 8. **Python Connector script** that connects with key-pair auth, runs a management query, and pulls a small dashboard result.
>
> If all eight parts work together, you have a complete real-time analytics platform: ingest, orchestration, transformation, ML scoring, security, and programmatic access - all in

Snowflake, with a Python client as thin as it can be.

SNOWPRO CORE ALIGNMENT

Chapter 4 contributes to four exam domains:

- **Data Loading and Unloading** - Streams, Tasks orchestration.
- **Data Transformations** - UDFs, stored procedures, dynamic SQL.
- **Performance Concepts** - Search Optimization Service.
- **Snowflake Data Cloud Features and Architecture** - the full ecosystem view.

Expect 10-14 questions on Chapter 4 material - the operational and application-integration content.

WHAT'S NEXT

In **Chapter 5 - ML, Cortex, and Certification**, we bring everything home. **Snowpark ML** trains and deploys models entirely inside Snowflake.

Cortex LLM Functions invoke ready-made large language models with a single SQL call. **Cortex Vector Search + Embeddings** implements RAG (Retrieval-Augmented Generation) natively - directly relevant to the Week 5 deep learning material on transformers and attention.

We also cover **cost optimization** patterns and finish with a comprehensive **SnowPro Core certification** review. Chapter 5 is the payoff chapter that ties Snowflake to the AI/ML curriculum you have already built.

SUMMARY CARD

Chapter 4 composed Snowflake's advanced features into production patterns: continuous CDC with Streams + Tasks, in-warehouse logic with UDFs + Stored Procedures, point-lookup acceleration with Search Optimization Service, and programmatic access via Python Connector + Snowpark. These are the everyday tools of senior Snowflake engineers.

Chapter 5 completes the journey with Cortex ML and certification review.